

LAGRANGIAN: Safely Optimizing Fully Homomorphic Encryption Computations with Constrained RL

ANONYMOUS AUTHOR(S)

In cryptography, FHE (Fully Homomorphic Encryption) is a promising method that enables computation over encrypted data without decryption. Although FHE has made significant progress, its substantial computational cost remains a major obstacle. This is why many compilers have been proposed to optimize FHE code automatically. While such compilers have made significant progress in minimizing execution time, they do not guarantee noise budget compliance. In other words, they do not guarantee that their optimized code does not exceed the allowed noise budget, which may lead to the corruption of encrypted data. In this paper, we propose LAGRANGIAN, a novel compiler framework that leverages *budget-conditioned, constrained reinforcement learning* to automate FHE code optimization. Instead of relying on predefined heuristics or combinatorial search, our method trains an RL agent to learn an effective policy for applying a sequence of rewriting rules to *automatically vectorize* scalar FHE code while optimizing latency *subject to a user-specified noise budget*. The approach supports both structured and unstructured code and coordinates global trade-offs between packing/layout, rotations, and multiplicative depth within the feasible space of code optimizations. To train the agent, we synthesize a diverse dataset of computations using a Large Language Model (LLM). We integrate our proposed approach into the CHEHAB FHE compiler and evaluate it on a suite of benchmarks, comparing its performance against Coyote, a state-of-the-art vectorizing FHE compiler. The results show that our approach generates code that is 5.3× faster in execution, 2.54× lower in consumed noise, and 27.9× faster to compile (geometric means), while *enforcing budget compliance* on our benchmark suite.

1 Introduction

Fully Homomorphic Encryption (FHE) enables computations directly over encrypted data without decryption, preserving confidentiality end-to-end. However, during the execution of an FHE code, every FHE operation increases noise in the ciphertext (encrypted data), and when the accumulated noise exceeds a certain limit (noise budget), decryption fails. The primary driver of noise growth is the multiplicative depth of the FHE circuit (FHE program). Longer chains of multiplication consume the noise budget much faster than additions and rotation operations. While a bootstrapping operation can restore the noise budget, such an operation is orders of magnitude more computationally expensive than arithmetic operations. Therefore, the use of such an operation is minimized.

Because the noise budget exhaustion leads to decryption failure, feasibility is non-negotiable. Any computation that violates the noise budget is invalid regardless of its speed. This imposes a safety-first ordering of goals when optimizing FHE code: ensure decryptability by bounding cumulative noise, and only then consider latency reduction. Solutions that deliver speedups at the cost of budget violations are, by definition, incorrect and must be excluded from consideration.

Although FHE has made significant progress, its substantial computational cost remains a major obstacle. This is why many compilers have been proposed to optimize FHE code automatically [15, 17, 20, 21, 40, 43, 61]. Systems such as HECO and CHET focus on structured, loop-based programs and exploit SIMD packing effectively, but they do not directly vectorize arbitrary unstructured expressions [21, 61]. More recent work, such as Coyote [43] and Porcupine [17], extends coverage to unstructured code by formulating vectorization as search over a combinatorial design space: which subexpressions to pack, how to lay out data in wide ciphertext vectors, where to insert rotations and masks, etc.

While existing FHE compilers have made progress in optimizing code for latency, they do *not* treat a user-specified noise budget as a first-class constraint at compile time. Feasibility is typically achieved by inflating cryptographic parameters (longer modulus chains, larger ring degree) or, when available, by inserting bootstraps. In some deployment use cases, however, doing so is not

possible. Instead, *leveled FHE with fixed parameters* is the operative regime: platforms standardize a set of cryptographic parameters for security and throughput, and bootstrapping is considered too costly. In these settings, *budget compliance must be enforced a priori*, not recovered ex post. Consequently, a compiler that guarantees a fixed noise budget when required is required.

We propose a *constrained, budget-conditioned* optimization method that is driven by a user-specified budget B , where the compiler guarantees that cumulative noise does not exceed B . Concretely, we cast the problem of FHE code optimization as a problem of *constrained Reinforcement Learning (RL) conditioned on the noise budget*: the RL policy takes B as context and minimizes latency *subject to* the budget constraint. This yields a compiler that trades speed for *guaranteed decryptability* under fixed parameters.

More concretely, we propose LAGRANGIAN, a compiler that uses *constrained, budget-conditioned* reinforcement learning to optimize latency *subject to* a budget constraint. LAGRANGIAN guarantees that cumulative noise does not exceed the budget constraint. It supports both structured and unstructured code and scales better than existing approaches. This is achieved through a policy network that selects a sequence of rewriting rules to vectorize code, while minimizing FHE operations and multiplicative depth, conditioning its decisions on B . Because our approach selects the sequence of rewriting rules using a policy network, it avoids computationally expensive search methods used in state-of-the-art FHE compilers, and therefore scales better. Under tight budgets the policy deliberately favors conservative transformations; we quantify this trade-off empirically while maintaining *zero budget violations*.

We integrate our proposed approach into the CHEHAB FHE compiler and evaluate it on a set of kernels, comparing it to Coyote, a state-of-the-art FHE compiler, and demonstrate that it produces higher-quality code and scales better.

The contributions of the paper are as follows:

- (1) We formulate FHE compilation as *constrained* sequential decision making: minimize latency *subject to* a user-specified noise-budget constraint.
- (2) We present LAGRANGIAN, a budget-conditioned, constrained-RL compiler that *automatically vectorizes* both structured and unstructured FHE code. It minimizes noise growth and circuit latency while scaling better compared to state-of-the-art, and while conditioning its decisions on the user-specified noise budget. To the best of our knowledge, this is the first FHE compiler to expose a user noise budget as a first-class knob and enforce it via budget-conditioned constrained RL.
- (3) Empirically, compared to Coyote, LAGRANGIAN generates code that is $5.3\times$ faster, consumes $2.54\times$ less noise, and requires $27.9\times$ less compilation time, while enforcing zero budget violations on our benchmark suite.

2 Overview and Motivation

2.1 Why Do We Need to Optimize FHE Code?

Despite notable recent progress and increasing practical adoption [22, 33, 37, 39, 61], FHE remains computationally expensive. For example, a ciphertext¹ multiplication takes 10^6 times longer than its plaintext counterpart². This disparity is due to the inherent complexity of homomorphic operations, as a single ciphertext multiplication entails multiple n -order polynomial multiplications. Reducing the execution time of FHE applications is essential for their practical adoption. Consequently, experts devote substantial effort to optimizing FHE implementations. Vectorizing FHE code while

¹Ciphertext: encrypted data; Plaintext: clear data.

²Evaluation done on the BFV scheme, for 128-bit security, using Microsoft SEAL [53] and running on a modern multicore CPU.

minimizing the number of FHE operations is one of the most important types of code optimization in FHE.

2.2 Example of Vectorizing FHE Code (Adapt CHEHAB Egg Example + Adapt to Safety/New illustrations)

In the example, show three cases: 1) optimizing for speed only (which breaks noise); 2) optimizing while making sure you minimize noise only (and not consuming all the noise budget, which leaves performance on the table); 3) optimizing while respecting the noise budget and while consuming the whole noise budget (so you respect noise, while maximizing speedup).

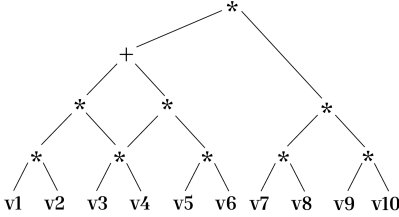
Vectorization in FHE is challenging as it introduces rotations, masking, and data packing. Poor vectorization can increase the number of rotations and circuit depth, severely impacting performance and increasing noise in the circuit. Thus, effective FHE vectorization must maximize code vectorization while minimizing rotation overhead and circuit depth.

Consider this unstructured (non-loop-based) code:

$$x = (((v_1 \cdot v_2) \cdot (v_3 \cdot v_4)) + ((v_3 \cdot v_4) \cdot (v_5 \cdot v_6))) \cdot ((v_7 \cdot v_8) \cdot (v_9 \cdot v_{10})) \quad (1)$$

Fig. 1a represents the code as a circuit (a common representation of code in the FHE literature). This expression is not trivial to vectorize.

To vectorize it, we use a Term Rewriting System (TRS) consisting of a set of rules. For illustration, we show only three rules, but the TRS has more.



(a) Example of an unstructured scalar code.

	$[1, 2, 3] = [v_1, v_5, v_6] \cdot [v_2, 1, 1]$
$[1, 2] = [v_1, v_5] \cdot [v_2, v_6]$	$[4, _] = [2, _] \cdot [3, _]$
$[3, _] = [1, _] + [2, _]$	$[5, _] = [1, _] + [4, _]$
$[4, _] = [v_3, _] \cdot [v_4, _]$	$[6, _] = [v_3, _] \cdot [v_4, _]$
$[5, _] = [3, _] \cdot [4, _]$	$[7, _] = [5, _] \cdot [6, _]$
$[6, 7] = [v_7, v_9] \cdot [v_8, v_{10}]$	$[8, 9, _] = [v_7, v_9, _] \cdot [v_8, v_{10}, _]$
$[8, _] = [6, _] \cdot [7, _]$	$[10, _] = [8, _] \cdot [9, _]$
$[x, _] = [5, _] \cdot [8, _]$	$[x, _] = [7, _] \cdot [10, _]$

(b) First vectorization. (c) Second vectorization.

Fig. 1. Motivating Example.

R1: multiplication_commutativity, $a \cdot b \Rightarrow b \cdot a$
R2: comm_factor, $a \cdot b + a \cdot c \Rightarrow a \cdot (b + c)$
R3: vectorization, $(a \cdot b) + (c \cdot d) \Rightarrow \text{Vec}(a, c) \cdot \text{Vec}(b, d)$

We first apply the rules R1 then R2, yielding:

$$x = ((v_3 \cdot v_4) \cdot ((v_1 \cdot v_2) + (v_5 \cdot v_6))) \cdot ((v_7 \cdot v_8) \cdot (v_9 \cdot v_{10})) \quad (2)$$

From this transformed expression, we can apply other rewriting rules to vectorize the code. There are many ways to vectorize it (also called vectorization strategies). We show two examples in Fig. 1 (intermediate values are named using numbers). Each vectorization strategy is obtained by applying a different sequence of rewriting rules. The key challenge is determining the sequence of rewriting rules that yields the best vectorization strategy while minimizing the number of rotations and depth of the circuit (multiplicative depth, more precisely, as we illustrate later).

We omit showing the rotation and masking instructions in the example for brevity. For example, in Fig. 1b, in order to calculate the operands of the 2nd statement ($[1, _]$ and $[2, _]$), rotations and masking should be applied on the result of the 1st statement as follows³:

$$\begin{aligned}[1, _] &= [1, 2] \ \& \ [F, _] \\ [2, _] &= ([1, 2] \ll 1) \ \& \ [F, _]\end{aligned}$$

but we omit showing these rotations and maskings.

Let us compare the two vectorizations in Fig. 1 using an approximate cost model that assigns a relative latency to each operation: multiplications and rotations have a latency of 1, while additions have a latency of 0.1 (these values are toy values chosen for simplicity only, to illustrate the example). The original scalar expression has 9 multiplications and 1 addition, yielding a total cost of 9.1. The vectorization in Fig. 1b reduces the number of multiplications to 6 and additions to 1 but introduces 2 rotations, resulting in a total cost of 8.1. In contrast, the vectorization in Fig. 1c involves 7 multiplications, 3 rotations, and 1 addition, yielding a 10.1 cost.

These results highlight that not all vectorizations are equally beneficial. Moreover, applying R1 prior to R2 is essential to obtain the expression in Eq. 2, which can then be vectorized by applying R3. In contrast, starting with R2 followed by R1 yields Eq. 1, where R3 is no longer applicable, thereby eliminating vectorization opportunities. This limitation highlights the importance of carefully selecting which rules to apply and in which order. To address this, we employ a policy network, trained via RL. The policy determines an ordered sequence of rules that optimize the code to achieve the best global vectorization, rather than relying on local improvements alone.

2.3 Why FHE Vectorization is Different from Classical Vectorization?

Vectorization in FHE is different from classical vectorization. While many code vectorization methods have been studied in the literature [7, 8, 13, 36, 38, 44, 57], such methods are not well-suited for vectorizing FHE code. First, since FHE does not support loops or conditionals, computations must be expressed as arithmetic circuits (unstructured code), preventing the direct use of classical loop-based vectorization methods. FHE vectorization differs from classical SLP-based approaches (Superword-level parallelism) [13, 38, 44] due to the large vector sizes in FHE (thousands of slots vs a few in hardware registers) and the inability to index ciphertext vectors directly. Instead, expensive rotation operations simulate indexed accesses. In addition, rotations in FHE are computationally expensive and increase noise in the circuit, so vectorization strategies must minimize their use. These differences make classical SLP-based methods unsuitable for FHE, as we discuss in Sec. 8. Because ciphertext vectors are wider in FHE compared to traditional hardware vector registers, FHE vectorization yields significant speedups because a single ciphertext operation (add or multiply) applies element-wise to every packed slot in the ciphertext, so one operation can replace thousands of scalar steps, far beyond the 4–8-lane limits of hardware SIMD.

2.4 The Noise-Budget Problem

In FHE, every homomorphic operation consumes a part of a finite *noise budget*; decryption is correct only while the remaining budget stays positive. For example, in a typical BFV configuration at 128-bit security, the budget begins around 360–370 bits. Each time an operation, especially a ciphertext–ciphertext multiplication, is run, it spends part of this budget. The dominant driver is multiplicative depth: long chains of multiplications rapidly deplete the budget even when the instruction count is small. In addition, alignment steps such as rotations and masking inject noise and often induce extra realignments or multiplications downstream; these non-local effects mean

³We assume 4-bit numbers in the example for simplicity, and we use $_$ to denote a zero-valued slot.

that a packing or layout that looks cheap locally can increase effective depth later. Additions and plaintext–ciphertext multiplications are comparatively benign, but their contributions accumulate. In this regime, the budget must be treated as a hard resource. Schedules that minimize latency can be fast, but they might exhaust their noise budget.

State-of-the-art compilers typically minimize latency (and sometimes proxies for noise), which reduces noise but does not guarantee noise budget compliance. First, feasibility is checked post hoc, so the search can spend substantial effort on candidates that are invalid under the target pack. Second, insisting on *noise minimization* can miss the best feasible design: solutions that are not noise-minimal but still *respect* the budget and deliver better latency within the feasible set. Given the global couplings introduced by vectorization and alignment, the budget must be treated as a first-class constraint—not a penalty term or after-the-fact filter—if we want reliable, efficient compilation.

2.5 Optimization Under a Fixed Noise Budget

Fixing the parameters is desirable in many cases as it greatly simplifies deployment. To illustrate this, let us consider the following example. Consider an FHE application running on a cloud service. If the parameter pack (set of parameters) is fixed, the client generates the public and evaluation keys once and reuses them across app updates: when the provider recompiles and ships a new binary, the user does not need to resend keys because keys depend only on the agreed parameters. This matters because FHE keys, especially rotation and relinearization keys, are large (often tens to hundreds of megabytes in aggregate); avoiding repeated key generation and upload reduces onboarding friction, bandwidth, and storage. Fixed packs also simplify compliance: the provider can publish a small set of pre-approved parameter packs that meet a recognized security level (e.g., 128-bit), perform a single formal review per pack, and then deem any app using that pack compliant. No per-app re-audit required. Operationally, a fixed pack enables predictable resource planning (ciphertext sizes and per-operation costs are stable), cacheable precomputations, and easy rollbacks (binaries compiled today and next month run under the same assumptions, and stored ciphertexts remain compatible). For these reasons, enabling a compiler to optimize code subject to a fixed budget is a valuable capability.

2.6 Our Approach at a Glance: Constrained RL for Budget-Respecting Rewrites

We formulate FHE compilation as a *constrained* sequential decision making: minimize latency *subject to* a user-specified noise-budget constraint. The state is the program IR (augmented with the remaining noise budget), an action is a *rewriting rule* that transforms the IR. We model this as *budget-conditioned constrained RL*: a single policy/value function is conditioned on the user-specified budget B , and is trained using a Lagrangian update together with a safety mask that blocks actions which would immediately violate the budget. The result is a compiler that *automatically vectorizes* structured and unstructured programs, coordinates global depth/rotation trade-offs, optimizes latency, while *respecting the noise budget*.

3 Preliminaries

3.1 Term Rewriting System (TRS)

A term rewriting system (TRS) is a set of rewrite rules that transform an expression to a new form (rewrite it into a new expression). Let us take a simple example of a TRS system to illustrate how it works. Let us assume that we want to simplify arithmetic expressions and let us assume that we have the following rule set $S = \{t - t \rightarrow 0, t + 0 \rightarrow t\}$. Let us use this rule set S to rewrite and simplify the expression $b + (a - a)$. The rule $t - t \rightarrow 0$ can be applied to simplify the expression

$a - a$. After applying the rule, we get the new expression: $b + 0$. Now we can apply the rule $t + 0 \rightarrow t$ to simplify the previous expression into b .

3.2 Reinforcement Learning

Reinforcement Learning (RL) is an area of machine learning where an *agent* learns to achieve a goal through trial and error by interacting with an environment [56]. At each step, the agent observes a *state*, performs an *action*, and receives a *reward*. The agent's goal is to learn a *policy*, which is a strategy for choosing actions that maximize the total cumulative reward over time.

3.3 Fully Homomorphic Encryption

FHE allows computations on encrypted data directly (i.e., without decrypting the data). For a function f and a message m , FHE allows the possibility of computing $\hat{y} = f(\hat{m})$, without knowing m . Here, $\hat{m}$ represents the encryption of m , and $\hat{y}$ is the encrypted output, which must be decrypted by the owner of m to obtain the output $\hat{y} \mapsto y$ in plaintext. FHE hence helps to achieve privacy-preserving computation in situations where sensitive data needs to be shared with third parties for computation offloading.

3.3.1 BFV Scheme. The Brakerski/Fan-Vercauteren (BFV) encryption scheme [9, 27] is an FHE scheme that provides primitives for modular addition and multiplication along with primitives for noise management and ciphertext maintenance (i.e., relinearization, modulus switching, and bootstrapping). BFV operates within plaintext and ciphertext spaces. Plaintexts and ciphertexts are large order polynomials defined by the parameters $\{n, t, q\}$. The degree n corresponds to the polynomial modulus $x^n + 1$, while the parameters t and q are the plaintext modulus and ciphertext modulus.

3.3.2 Encryption Parameters for the BFV Scheme. Encryption introduces *noise* for security, which grows with operations on ciphertexts or between a ciphertext and a plaintext. Its growth is limited by q and t (defined in A.1.1); exceeding this limit corrupts the data, leading to incorrect decryption (a user sets a noise budget for their circuit, and the circuit has to run without exceeding that noise budget). *Bootstrapping* can reset noise but has high computational costs. Instead, FHE applications typically limit operations to a predefined depth. Supporting deeper circuits requires increasing q and adjusting n for security. This is especially important for circuits with high multiplicative depth, as multiplications cause exponential noise growth, while additions increase it linearly. However, larger q and n also introduce performance overheads due to larger polynomials and coefficients. Therefore, one of the goals when optimizing an FHE circuit is to minimize the multiplicative depth to be able to use the smallest q and n that still respect the noise budget in the circuit. This, in turn, makes FHE operations run faster and reduces the overall circuit latency.

3.3.3 Vectorization in FHE. Vectorization in FHE (a.k.a., *batching*) is a technique that encodes a vector of n integers in a single plaintext polynomial using the Chinese Remainder Theorem (CRT). Vectors in FHE are large (thousands of slots). With vectorization, additions and multiplications are executed slot-wise in a Single-Instruction Multiple-Data (SIMD) fashion. Moreover, BFV supports rotations of the encoded vector to move data (and perform data packing). Rotation as a function takes a ciphertext (vector) and a rotation step and returns the ciphertext (vector) with slots shifted in a cyclic manner. For example, the rotation $[1, 2, 3] \ll 1$ returns $[2, 3, 1]$. Note that FHE supports only additions, multiplications, and rotations and does not support branches or loops. See Appendix A.1 for a formal definition of FHE concepts.

3.3.4 Circuit Depth and Multiplicative Depth.

Circuit Depth. An FHE program can also be represented as a circuit. A circuit is a DAG (Directed Acyclic Graph) representation of the program that captures its dataflow. Calculating the depth of the circuit is important for estimating noise. The depth of a circuit (DAG) represents the maximum number of consecutive operations performed on an input to compute the output. The depth of a node v in the DAG is defined as the length of the longest path between v and an input node. This depth is usually used to quantify the noise in the circuit. However, it does not provide any information on the type of operations involved (which is necessary to determine how the noise grows). Since multiplication has the most significant impact on noise, one should compute the multiplicative depth of the circuit to estimate noise better.

Multiplicative Depth of a Circuit. Similar to circuit depth, except that we consider counting the number of multiplications only, since multiplications increase noise exponentially. The multiplicative depth of a node v is defined as the length of the longest path, when counting multiplication operations alone, between v and an input node.

4 Compiler Overview

We integrate LAGRANGIAN as a pass in the CHEHAB compiler. CHEHAB defines a domain-specific language (DSL) for FHE. The CHEHAB DSL is the compiler’s starting point. An intermediate representation (IR) is first generated from the CHEHAB DSL. This IR is in the form of an AST (Abstract Syntax Tree). The IR is then optimized by LAGRANGIAN, our reinforcement learning (RL) agent, which learns a policy to apply a sequence of transformations as described in Sec. 5. These transformations are drawn from a set of rewriting rules that include rules for both code vectorization and for simplification to reduce noise growth and instruction latency (i.e., rewrite complex arithmetic expressions into simpler expressions that have a smaller number of operations and depth). To further optimize code, the CHEHAB compiler applies classic compiler optimizations, including common subexpression elimination (CSE), constant folding, and dead code elimination. Finally, the code generator generates a sequence of vectorized operations implemented in C++ and targeting Microsoft SEAL’s backend [53] for BFV. Our main contribution in this paper is the introduction of a Term Rewriting System that uses constrained reinforcement learning to select the rewriting rules to apply while respecting the noise budget. We call this the RL TRS.

The CHEHAB DSL is a lightweight DSL, embedded in C++, that lets developers write FHE programs at a high level while the compiler manages the low-level cryptographic details. Instead of hand-tuning encryption parameters and rotation keys, a time-consuming task that demands specialized expertise, developers express computations with familiar operators over Plaintext and Cipher text types, and CHEHAB automatically lowers the code to the FHE paradigm, generating and managing all required FHE artifacts (e.g., public and rotation keys) on their behalf.

We present the CHEHAB compiler, the CHEHAB DSL, the CHEHAB IR, and the compilation pipeline in Appendix B.

5 Problem Formulation as a Constrained Reinforcement Learning Problem

Bilel: Add a figure for each RL component

The core of the CHEHAB RL framework is a Term Rewriting System (TRS) that uses RL to transform an initial Intermediate Representation (IR) into an efficient one. We frame the optimization task as a sequential decision-making problem, formally a Markov Decision Process (MDP), where an RL agent learns a policy to select a sequence of rewriting rules from a set of predefined rules. The policy is designed to select the sequence of rewriting rules that minimizes latency and noise. It can select rewriting rules that initially yield low rewards, but when combined with additional

rules, result in higher long-term rewards. This approach allows the agent to automatically navigate the complex trade-offs between vectorization, rotations, and circuit depth.

Our RL framework uses the actor-critic RL architecture and is defined by the following key components: 1) *State representation* (detailed in Sec. 5.2): provides a representation of the program being optimized; 2) *Action space* (Sec. 5.3): defines the set of rewriting rules that the agent can apply to the program being optimized; 3) *Reward model* (Sec. 5.4): provides a function that scores the quality of actions, guiding the agent toward actions that produce FHE circuits that are more efficient and have less noise; 4) *Actor-critic networks* (Sec. 5.5): the actor selects the next rewriting rule to apply given the current program state, and selects where to apply it, while the critic estimates the expected return and provides the learning signal for policy optimization during training (the critic is only used during training to train the policy).

We train our proposed RL agent on an *LLM-generated dataset* (described in Sec. 6) using the *PPO learning algorithm* (Sec. 7.1.2). The following sections describe these components.

5.1 Constrained Reinforcement Learning Formulation (Ilyas: Write)

[TODO]

5.2 State Representation

The goal of the state representation is to produce a fixed-length vector embedding, s_P , that represents a given program IR, P . To extract the program representation, we first tokenize the program being optimized. We use a simple method that we call *Identifier and Constant Invariant (ICI)* tokenization, which produces a canonical token sequence $T(P)$ that is invariant to identifier names and to the concrete values of constants. For each program P , we create a per-program renaming: the first distinct variable is mapped to v_0 , the second to v_1 , and so on. This method has two advantages: it creates the same tokenization for programs that are semantically equivalent but only differ in their variable names or constant values. This enables the model to treat them in the same way, simplifying learning for the agent and enabling it to generalize better. Second, this method is fast and straightforward compared to classical Byte-Pair Encoding (BPE) tokenization [54]. BPE is slower because it repeatedly applies merge rules and performs large-vocabulary prefix searches; by contrast, identifier and constant invariant tokenization is a single left-to-right pass with $O(1)$ hash-map updates per new identifier. Therefore, it is much faster (more details about this tokenization method in Appendix D).

After tokenization, the program P is represented by a variable-length sequence of tokens, $T(P) = [t_1, t_2, \dots, t_k]$. We then convert this sequence into a fixed-length vector that captures the long-range dependencies and hierarchical structure implied by the tokens. For this, we use a Transformer encoder [59] that learns automatically a short, fixed-length embedding for the tokenized program. We chose this architecture because its self-attention mechanism is designed to model long-distance dependencies inherent in source code, a capability validated by state-of-the-art code models such as CodeBERT [28]. This is critical for processing the long token sequences that represent unrolled FHE programs. Early in the project, we compared the Transformer architecture to recurrent architectures, including LSTMs and GRUs, but they did not provide satisfactory results. The Transformer encoder follows a standard architecture, consisting of a stack of 4 identical layers. Each layer employs a multi-head attention mechanism with 8 attention heads. The input token embeddings are summed with absolute positional encodings to preserve sequence order, and the encoder processes this sequence to generate a 256-dimensional vector for each token (we use the CLS special token as a placeholder to summarize the entire input sequence, as is commonly done in the literature [23]).

5.3 Action Space

The action space defines the set of valid transformations the agent can perform on the current program state. In our framework, an action corresponds to a rewriting rule selected from a set of predefined rewriting rules.

A challenge in designing the action space is handling the location of rule application. A single rule might match multiple sub-expressions within the IR. To address this, the agent needs to select both a rule and a location. To enable this, we propose a hierarchical action space composed of two steps: the agent first picks a rule to apply, then picks the location where to apply it. To implement this, we propose a *policy network* used to pick an action, and a *location network* used to pick a location.

To allow the agent to control the length of the optimization process, the action space also includes a special END action. Selecting this action terminates the current optimization episode, allowing the agent to learn not only which rules to apply, but also when to stop.

In total, our TRS consists of 84 rewrite rules, in addition to the END action. To design these rules, we began by collecting rules from Halide’s TRS [46, 50], and excluded rules that are not compatible with FHE (comparison, division, etc.). We then added new rules designed to reduce circuit depth, multiplicative depth, rotations, and the total number of operations. More details about the rewriting rules in Appendix C.

5.4 Reward Model

The agent learns via a reward signal designed to guide it toward producing FHE circuits that have minimal latency and noise. This signal is derived from an FHE-aware cost function that evaluates the quality of a given IR. We will first show the cost function, and then show how it is used to compute the reward.

5.4.1 FHE Cost Function. The goal of this cost function is to capture the metrics that we aim to minimize: number of FHE operations, multiplicative depth, and depth of the circuit. We also want to minimize the use of scalar operations in favor of vector operations. By minimizing the previous metrics, we minimize latency and noise. To achieve our goal, we define the cost function of an IR expression, e , as the weighted sum of the metrics that we want to minimize:

$$\text{Cost}(e) = w_{\text{ops}} \cdot C_{\text{ops}}(e) + w_{\text{depth}} \cdot D_{\text{circuit}}(e) + w_{\text{mult}} \cdot D_{\text{mult}}(e)$$

Where C_{ops} is the cost of operations, D_{circuit} is its circuit depth, and D_{mult} is its multiplicative depth. The weights w_{ops} , w_{depth} , and w_{mult} reflect the importance of each metric in the cost function. For our experiments, we set all weights to 1. The components of this cost function are defined below.

Operations Cost (C_{ops}): This term quantifies the computational cost of all of the operators used in the expression. We assign a numerical cost to each operator. These numerical costs reflect its relative performance compared to other operators in the BFV scheme. The relative order of these operations is guided by former empirical studies on the cost of FHE operations [34]. The costs are structured to incentivize vectorization: 1) *Vector Additions/Subtractions*: These have the lowest cost (1), as they represent the most efficient parallel operations. 2) *Vector Multiplications*: These have a higher cost than vector additions/subtractions, and usually cost 100× the cost of vector additions, and therefore we assign a cost of (100) to them. 3) *Rotations* ($\llcorner$): Rotations typically have a cost that is 0.5× to 1× of the cost of a vector multiplication (depending on the parameters). We assign a cost of (50) to rotations, reflecting that rotations might be cheaper than multiplications but always more expensive than additions. This incentivizes rotations over multiplications, since they have

a cost that is usually less than or equal to the cost of multiplications. 4) *Scalar Operations* (+, -, *): These are assigned a high cost (250) to reflect their inefficiency. While in this particular case, the value 250 is not derived empirically, we chose this high cost for scalar operations to incentivize the RL to generate vectorized code.

The total C_{ops} is the sum of the costs of all the nodes in the expression tree.

Circuit Depth (D_{circuit}). A greater depth implies more sequential operations are performed on a ciphertext, which leads to a larger accumulation of noise. Minimizing depth is therefore critical, as it directly impacts the noise.

Multiplicative Depth (D_{mult}): This metric is critical as noise growth in the BFV scheme is directly proportional to the multiplicative depth. Therefore, to minimize the noise, we aim to minimize the multiplicative depth.

5.4.2 Reward. The reward is given at the end of an episode (when the agent selects the END action, or the pre-defined maximum steps limit for the episode is reached). This reward is based on the total percentage reduction in cost from the initial state of the expression (C_{initial}) to its final state (C_{final}):

$$R_{\text{final}} = \left(\frac{C_{\text{initial}} - C_{\text{final}}}{C_{\text{initial}}} \right) \times 100$$

We have also tried another reward structure where intermediate rewards are provided after each step, in addition to the above terminal reward, but that did not help in improving the quality of the policy.

5.5 Actor-Critic Networks

In this work, we adopt the actor-critic RL architecture. To manage the complexity of the action space, we learn a *hierarchical stochastic policy* (actor), which decomposes the action a into two components: a rewrite rule r and an application location p . This policy is defined by two conditional probability distributions: $\pi_{\theta_1}(r \mid s)$ for rule selection and $\pi_{\theta_2}(p \mid s, r)$ for location selection. Alongside the policy, we learn a value function (critic) $V_{\phi}(s)$, which estimates the expected return from a given state s . The policy and critic networks are implemented as separate deep neural networks.

5.5.1 Rule Selection Network. The first network of the actor is the rule selection network. It is a Multi-Layer Perceptron (MLP) that takes, as input, the 256-dimensional state embedding generated by the Transformer encoder (Section 5.2). The network consists of two fully connected hidden layers with 128 and 64 neurons, respectively, each using the ReLU activation function. The network’s final output layer is followed by a softmax, which produces a probability for each available action from the action space. After invalid actions (non-matching rules) are masked, the agent samples from this resulting probability distribution to select its next action.

5.5.2 Location Selection Network. Once the rule network selects a rewriting rule, the Location Network selects the location within the IR for its application. The network selects whether the rule is applied to its 1st match in the IR, 2nd match, 3rd match, etc. It is implemented as an MLP that takes two inputs: the 256-dimensional state embedding and the output from the actor network representing the chosen rule. Its architecture consists of two fully connected hidden layers, each with 64 neurons, followed by ReLU activation. The final output layer produces a probability distribution over locations, from which the final position is sampled.

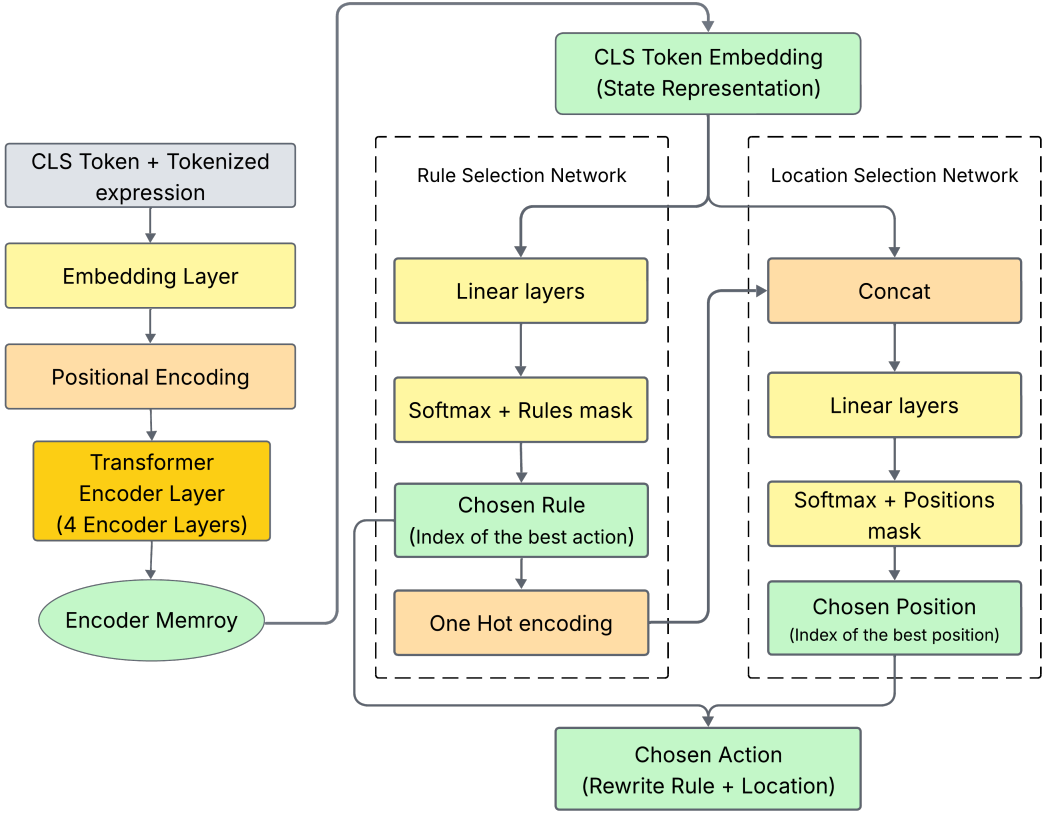


Fig. 2. Architecture of the Policy Network (A New Figure)

5.5.3 The Critic (Value Network). The critic network estimates the value function, $V(s)$, which predicts the expected cumulative future reward from the current state. It is an MLP that takes the state embedding as input. Its architecture consists of 3 fully-connected hidden layers with 256, 128, and 64 neurons, respectively, each with a ReLU. The final layer is a linear output layer that produces a single scalar value representing the value estimate, $V(s)$. This estimate is used during training to assess the actor's actions and guide the learning.

6 Synthesizing the Training Dataset **Rephrase.**

A key challenge for training an effective RL agent is generating a dataset of expressions that are both diverse and contain realistic code patterns that appear in practice

6.1 Why Randomly Generating Data is Not Ideal?

Training on purely random expressions is not ideal. Randomly generated expressions do not follow the same distribution of programs encountered in practice. They often lack the specific computational patterns that appear in practice (e.g., common sub-expressions, opportunities for factorization, and structured arithmetic that can be vectorized).

6.2 Using LLMs to Synthesize Data

To address the previous limitation, we use *Large Language Models (LLMs)* to generate the training data. We guide the LLM to synthesize a dataset that is diverse and rich in optimizable code patterns. Since the LLM is trained on a large corpus of realistic code, it tends to generate code that follows the same distribution of realistic code. For our data generation, we used the *Gemini 2.5 Flash* model. The prompt provided to the LLM is structured to give it sufficient context about the problem domain, and includes:

- i) *Syntax and Semantics*: A formal description and multiple examples of the CHEHAB IR, covering scalar operations, vector constructors (Vec), and vector operations (e.g., VecAdd).
- ii) *Rewrite Rule Examples*: Examples of the rewrite rules from our TRS are included to guide the LLM on the specific patterns we intend to optimize.
- iii) *Real World Examples*: To ground the synthesis process in practical computations, the prompt contains real-world kernels as examples. Examples provided in the prompt include: a) Union Cardinality: Computes the cardinality of the bitwise OR of two 4-bit vectors; b) Squared Difference: Computes the element-wise squared error between two vectors; c) 4x4 Matrix Addition: A common linear algebra kernel that performs element-wise addition between two matrices. The provided examples do not include any of the benchmarks used in the evaluation.
- iv) *High-Level Goal*: A description of the optimization objective and an explicit request for structural diversity in the generated expressions.

Post Processing. Expressions generated by the LLM are filtered to ensure their validity and uniqueness:

- i) *Parsing and Validation*: Expressions that cannot be parsed by our IR parser (due to syntactic errors) are discarded.
- ii) *Uniqueness Filtering*: To ensure diversity and avoid redundancy, we implement a uniqueness filter based on the canonical representation of expressions. This canonical representation is obtained using our Identifier and Constant Invariant (ICI) tokenization (described in Appendix D). Generated expressions may be semantically equivalent but use different variable names (e.g., $(\text{Vec } (+ \times (* y z)))$ and $(\text{Vec } (+ a (* b c)))$). After ICI tokenization, they become equivalent. A newly generated expression is added to the dataset only if its canonical form was not generated before.
- iii) *Exclusion of Benchmarks*: To avoid training the RL agent on programs identical to those found in the benchmark, we remove any generated expression that is identical to one of the benchmarks. To do this, we compute the canonical form of each program in our benchmark suite (Sec. 7) using ICI tokenization. Any expression in the generated dataset that matches a benchmark is removed.

Our approach ensures that the dataset is not only syntactically correct but also semantically relevant and rich in the patterns our agent needs to learn to optimize. We generated 15,855 unique expressions using the LLM and used this dataset to train the RL agent.

7 Experiments (Update/Rephrase/Move all Results from the Appendix)

7.1 Experimental Setup

7.1.1 Hardware. We evaluate our benchmarks on a 2.40 GHz Intel(R) Xeon(R) CPU E5-2680 v4 with 256 GB DDR4 of memory running CentOS Linux 8. We also use the same hardware for training the RL agent.

7.1.2 RL Agent Training. We use the Proximal Policy Optimization (PPO) algorithm to train the agent [52] used by CHEHAB RL. We train it for 2 million timesteps (43 hours), on a single multi-core CPU. More details about the training, and hyperparameters, in Appendix E.

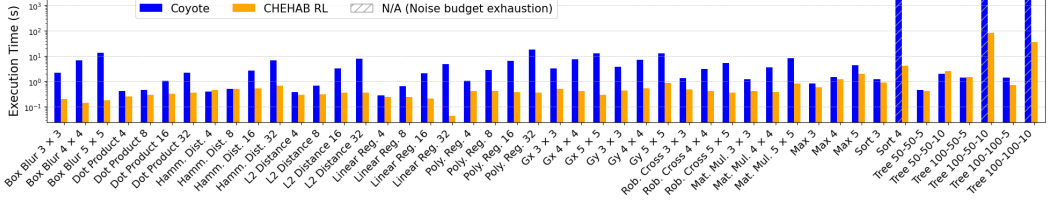


Fig. 3. Semi-log plot of execution time of the benchmarks, comparing code generated using our compiler and Coyote. (Change Colors)

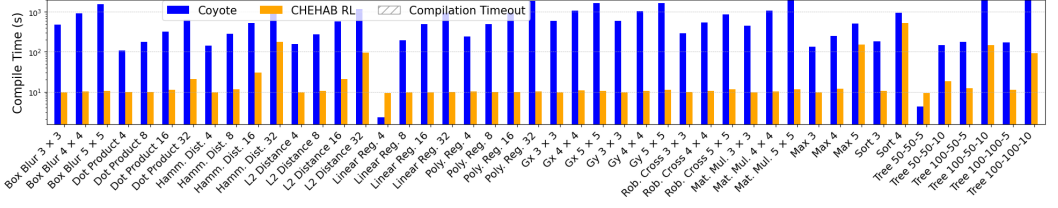


Fig. 4. Semi-log plot of compilation time of the benchmarks using our compiler and Coyote.

7.1.3 Environment Setup. The encryption parameters for CHEHAB and Coyote are set as follows: $n = 16384$ (polynomial modulus degree) and q is the default coefficient modulus proposed by the SEAL library for a standard 128-bit security level. We also use the same SEAL library in both compilers (SEAL version 4.1). We run all the experiments 30 times and report the median execution times and compilation times. The compilation timeout for both compilers was 7200 seconds.

7.2 Evaluation Objectives

We evaluate our proposed approach in three ways:

Quality of Programs Generated. We compare code generated by CHEHAB RL with that produced by the Coyote [43] compiler. The comparison focuses on the execution time of the generated circuit, the number of homomorphic operations, the depth, the multiplicative depth, and noise (noise consumed by the benchmark, which we want to minimize). The definition of the noise metric and details about how we measure it are in Appendix F.1. We limit our evaluation to FHE compilers that support unstructured code. Porcupine [17] is excluded since it has not been released by its authors.

Compilation Efficiency. We report the end-to-end compilation time for CHEHAB RL and Coyote.

Ablation Study. We perform an ablation study to evaluate the following: 1) LLM-generated vs random data; 2) CHEHAB RL vs CHEHAB; 3) ICI vs BPE tokenization; 4) Flat vs hierarchical action space; 5) GRU vs Transformer encoder.

7.3 Baseline Compilers

We compare code optimized by LAGRANGIAN to code optimized by the Coyote [43] compiler. We use the default parameters for the Coyote compiler.

We tried to enable the blocking technique in Coyote [43] but that led to compiler errors in Coyote, therefore we do not use this technique in both compilers, CHEHAB and Coyote, to have a fair comparison. Blocking is a technique that involves vectorizing smaller kernels separately and then

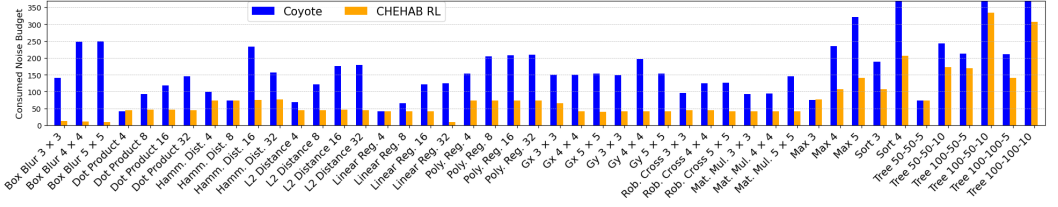


Fig. 5. Semi-log plots comparing the Consumed Noise in Coyote and CHEHAB RL.

composing the resulting vectorized programs. While this technique enables the two compilers to scale to larger programs, it is orthogonal to our contribution. Our goal is to compare the fundamental algorithms proposed by each compiler on a single block. Since the blocking method is identical in the two compilers, and since our contribution is orthogonal to this technique, comparing on a single block better illustrates the fundamental differences between the two approaches.

Our comparison is limited to Coyote because no other compiler, except Porcupine, supports both structured and unstructured code. Comparing with other tools will not help in evaluating our research question, as they only support structured code. Regarding Porcupine, it has not been released. We reached out to the authors of Porcupine to ask for a copy of the compiler but unfortunately we could not obtain a copy. The only released compiler that supports unstructured code, for now, is Coyote.

7.4 Evaluation Benchmarks

We evaluate CHEHAB RL on three benchmark suites. For each benchmark in the benchmark suites, we vary the input size to evaluate the scalability of the compiler. The larger the input is, the larger the program is, since FHE code is fully unrolled. The three benchmark suites are:

Porcupine Benchmark Suite. This is a set of kernels commonly used in linear algebra, machine learning, and image processing. They were used to evaluate Porcupine [17]. They include image processing filters: Box Blur, the x and y gradients of an image (Gx and Gy), and Robert Cross (*Rob. Cross.*). The Porcupine benchmark suite also includes Dot Product, Hamming Distance (*Hamm. Dist.*), L2 Distance, Linear Regression (*Linear Reg.*), and Polynomial Regression (*Poly. Reg.*), which are building blocks for ML applications. We include all of the benchmarks that were used to evaluate Porcupine.

Coyote Benchmark Suite. It includes kernels that were used to evaluate Coyote [43]. These kernels are: 1) matrix multiplication (*Mat. Mul.*); 2) a sorting algorithm (*Sort*) that sorts a list of elements; This sorting algorithm is an unstructured code that implements sorting using a tree (as described by Malik et al. [43]). 3) *Max* kernel. This is an unstructured code that finds the maximum element in a list. Coyote also includes three other kernels, which are the dot product, the L2 distance, and a convolution, but these are common with the Porcupine benchmark suite, and therefore, we do not evaluate them again. All of the benchmarks that were used to evaluate Coyote are included in our benchmark suite.

Randomly Generated Irregular Polynomials. The Coyote compiler also uses a set of randomly generated unstructured polynomials for evaluation. We use the same methodology for evaluation. This set of polynomials is a stress test to further investigate the ability of the compiler to vectorize code in the absence of a regular structure in the code. Several polynomials are generated as random

arithmetic expression trees. In these polynomials, a polynomial is named tree-X-Y-Z, where high values for X-Y indicate that the polynomial tree is full, complete, and well-balanced, making it easier to vectorize, while low values for X-Y indicate that the tree is sparse and imbalanced, making it harder to vectorize. Z indicates the depth of the tree of the polynomial. We provide more details about these polynomials in Appendix F.3.

7.5 Results of the Evaluation on the Benchmarks **Mounir: Update the figures, and the discussion in the text; rephrase the text**

We compare CHEHAB RL with Coyote on our benchmark suite, measuring the execution time of optimized code (Fig. 3), compilation time (Fig. 4), and consumed noise (Fig. 5). Fig. 3 shows that CHEHAB RL produces faster code than Coyote for most benchmarks. On average, CHEHAB RL generates code that is $5.3\times$ faster in execution (geometric mean) than Coyote. For example, on Poly. Reg. 32, code generated by CHEHAB RL is $50\times$ faster than code generated by Coyote, and faster by $114\times$ on Linear Reg. 32. This speedup is due to having fewer operations in code generated by CHEHAB RL, for example, in Poly. Reg. 32, CHEHAB RL generated a circuit with just 3 additions and 2 ct-ct multiplications (ciphertext-ciphertext multiplications). In contrast, Coyote generates a much larger circuit with 12 additions, 2 ct-ct multiplications, 1 subtraction, 173 ct-pt multiplications (ciphertext-plaintext multiplications), and 134 rotations. Coyote appears to generate a complex data layout that requires extensive rotations and ct-pt multiplications to execute, where CHEHAB RL finds a simpler, more direct vectorization. This pattern holds for Linear Reg. 32 as well, where the agent produces a circuit with 2 additions, 1 ct-ct multiplication, and 1 ct-pt multiplication, while Coyote generates one with 4 additions, 1 ct-ct multiplication, 3 subtractions, 46 ct-pt multiplications, and 44 rotations. Image-processing kernels also show substantial gains: code generated by CHEHAB RL for Gx 5x5 is $42\times$ faster than Coyote. In this benchmark, Coyote over-rotates data, leading to a higher number of operations and a slower circuit. For Tree 50-50-10, Coyote generates a circuit that runs faster; this is due to the high number of expensive ct-ct multiplications that CHEHAB RL generates. The generated circuits that Coyote generates, despite having a higher number of total operations (21 adds, 19 plaintext multiplications, 7 rotations), require only 7 ct-ct multiplications. In contrast, the circuit produced by CHEHAB RL, while more compact (18 adds, 2 rotations), requires 15 ct-ct multiplications.

Fig. 4 shows that despite producing circuits that are faster to execute, the compilation time for CHEHAB RL is also consistently faster than Coyote’s. On average, the CHEHAB RL compilation process is $27.9\times$ faster (geometric mean) than Coyote. A notable exception to this trend is the Tree 50-50-5 and Linear Reg. 4 benchmarks, where Coyote’s compilation time is faster than CHEHAB RL. These benchmarks are small, so Coyote’s search algorithm can rapidly explore the small search space. In contrast, our RL agent requires a series of steps to apply its learned policy. The overhead of invoking the neural network for each of these sequential steps results in a longer compilation time for these small benchmarks. However, as demonstrated across the rest of the benchmark suite, this overhead for the RL agent is outweighed by its higher scalability to larger circuits.

Fig. 5 compares the noise consumed by the circuits generated by CHEHAB RL and Coyote. The results show that code generated by CHEHAB RL consistently consumes less noise compared to Coyote. On average, code generated by CHEHAB RL consumes $2.54\times$ less noise (geometric mean) compared to Coyote. For example, code generated by CHEHAB RL for Poly. Reg. 32 consumes only 73 bits from the available noise budget (which is 369 bits), leaving a remaining budget of 296 bits, while Coyote’s much larger and complex circuit consumes 210 bits. In different benchmarks such as Sort 4 and two of the Polynomial Tree benchmarks, the circuit generated by Coyote

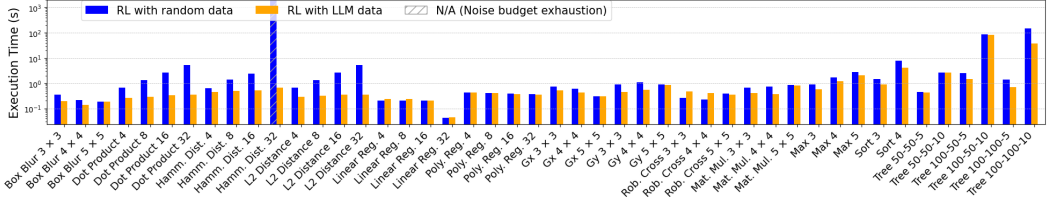


Fig. 6. Semi-log plots comparing the execution times when using an RL agent trained with randomly generated data and an RL agent trained with data generated by an LLM

exhausts the entire noise budget and fails to execute, while CHEHAB RL successfully produces valid, runnable circuits for all benchmarks, leaving a safe remaining noise budget in each case.

A more detailed comparison, including operation counts for all benchmarks, is available in Table 5 in Appendix F.4.

7.6 Ablation Study **Mounir: Add the New Ablation Studies; rephrase the text.**

LLM-generated vs. Random Data. To quantify the effectiveness of using an LLM to generate data, we compare the performance of our RL agent (trained on LLM-generated data) against the same agent trained on a dataset of randomly generated data. The random code generator uses a standard approach to generate code with a uniform distribution (described in Appendix F.2). Fig. 6 compares the two approaches. The agent trained on LLM-generated data produces FHE circuits that are faster. For instance, on the *L2 Distance 32* benchmark, CHEHAB RL trained on LLM-generated data produces code that is 13× faster compared to CHEHAB RL trained on randomly generated data. Similar performance gaps of an order of magnitude are observed across the larger instances of *Hamm. Dist.* and *Dot Product*. This difference in performance is due to the quality of the learned policies. The agent trained on LLM-generated data learns to generate more efficient circuits with fewer expensive homomorphic operations. For example, in the *L2 Distance 32* case, the circuit generated by the LLM-trained agent contains only 5 additions and 1 multiplication, compared to 28 additions and 28 multiplications in the circuit generated by the agent trained on random data. This demonstrates that the LLM-generated data helps the RL agent learn a policy that performs better on realistic programs, since it provides expressions with relevant, optimizable structures, that more closely mirror real program distributions.

CHEHAB RL vs CHEHAB. To ensure that the improvements in reducing latency in CHEHAB RL are due to the RL TRS, and not to other optimizations in the original CHEHAB, we compare the execution times of code generated by the original CHEHAB and CHEHAB RL. Fig. 8 (Appendix) shows the results. As we see, in most cases, code generated by CHEHAB RL is faster. In some instances, most notably *Gx 3x3*, the original CHEHAB is better than CHEHAB RL due to a sub-optimal decision made by the learned policy. An examination of the generated circuit shows that the RL agent chose to apply one rotation to align data for the vectorization of two ct-pt multiplications. While the CHEHAB RL successfully vectorized the operations, the computational cost of the required rotation was greater than the performance benefit gained from vectorizing the two ct-pt multiplications.

ICI vs. BPE Tokenization. We also evaluate the use of the Identifier and Constant Invariant (ICI) tokenization for tokenizing the inputs of the embedding model. This experiment compares our agent, which uses ICI tokenization, against the same agent that uses a standard Byte-Pair Encoding (BPE) tokenizer. The BPE tokenizer was trained on a corpus of 5 million randomly generated IR

expressions to build its vocabulary (details about random IR code generation are in Appendix F.2). Once the BPE tokenizer is trained, we train the two agents using our dataset of LLM-generated data. Fig. 9 in the appendix shows that the RL agent that uses ICI tokenization finishes its 2 million steps training in 43 hours, compared to the RL agent trained with BPE, which takes 68 hours, showing that our tokenization helps accelerate the training.

Flat vs. Hierarchical Action Spaces. We also compared the hierarchical action space against a flat one that enumerates rule-location pairs (i.e., each rule is duplicated multiple times, once for each location). The learning curves in Fig. 10 in the appendix show consistently higher rewards and quicker learning for the hierarchical agent.

GRU vs Transformer. To validate our choice of a Transformer encoder for the state representation, we compared its encoding capabilities against a baseline recurrent architecture, the Gated Recurrent Unit (GRU). Our experiments show that the transformer architecture was able to learn a better representation with less training compute. Details about this experiment are in Appendix F.5.

8 Related Work

This section compares CHEHAB RL and state-of-the-art compilers. Table 1 summarizes the differences, which we discuss in detail in this section. In addition, we discuss the use of RL in compilers, tokenization methods, and random data generation methods in Appendix G.

FHE Compilers for Automatic Vectorization. Recent work [17, 21, 43, 61] focuses on the problem of transforming FHE programs that use scalar variables into vectorized programs (batching). Our work is similar to these compilers since it also takes a scalar code and vectorizes it. Unlike HECO [61] and CHET [21], ANT-ACE [41], Orion [25], HEIR [3] and Qiwu [64] which only support the vectorization of structured (loop-based) code, CHEHAB RL is designed to support both structured and unstructured code. Compared to Coyote [43], our approach scales better. Unlike Porcupine, in which the user has to manually provide the data layout of the vectorized code, our approach computes the best data layout automatically.

FHE Compilers for Circuit Optimization. These compilers do not vectorize their input code. They focus on applying other optimizations (other than vectorization). These compilers include EVA [20], Ramparts [4] and HECATE [40]. The main challenges that they target are automatic parameter selection, the scheduling of ciphertext-maintenance operations, reducing the multiplicative depth of circuits, and reducing the number of operations in a circuit. Our approach is complementary to the above compilers. First, it supports code vectorization, and therefore, code vectorized by our approach can be passed to these compilers for further optimization. Our approach also offers the ability to reduce the depth of circuits, which they do not address. Appendix G provides a more detailed comparison with these compilers.

General Purpose Vectorization in non-FHE Programs. Superword-Level Parallelism is a classical technique used for code vectorization [38]. It processes a sequence of scalar instructions to create vector packs or groups of isomorphic instructions that can be packed together into vectors. Since it does not depend on the presence of data-parallel loops in the code, it is well-suited for vectorizing unstructured code. However, while creating vector packs, SLP does not consider the high cost of rotations in FHE programs, which results in code that has a high number of rotations, making it impractical for the domain of FHE, where rotations are expensive. goSLP [44] is a state-of-the-art SLP approach that formulates the vectorization problem as an Integer Linear Programming (ILP) problem. However, it operates at a level closely tied to the target architecture, where the vectors are restricted to a maximum width of four elements. One of its key limitations is that while it efficiently

Table 1. COMPARISON WITH RELATED WORK.

	RL						
Feature	CHEHAB	HECO	Porcupine	Coyote	Ramparts	EVA	HECATE
Vec. struct. code	✓	✓	✓	✓	×	×	×
Vec. unstruct. code	✓	×	✓	✓	×	×	×
Reduce mul. depth	✓	✓	✓	×	✓	×	×
Auto datalayout	✓	✓	×	✓	✓	✓	✓
Data driven	✓	×	×	×	×	×	×
Scheme	BFV	BFV CKKS BGV	BFV	BFV	BFV	CKKS	CKKS

handles the pairwise packing of statements, extending this to pack more than two statements makes the problem intractable for current solvers. In contrast, our approach for FHE supports significantly larger vectors (in the thousands), which is necessary for the domain of FHE since vectors in FHE are significantly larger. VeGen [13] extends SLP by introducing lane-level parallelism, tracking how individual lanes execute computations. This enables VeGen to account for rotation costs when constructing vector packs. However, this reasoning is local, as VeGen does not consider the impact of instruction packing on subsequent rotations. Other work [26, 48, 51] also addresses code vectorization but is not designed for the field of FHE and does not assume a high cost for rotations. In addition, in this work, our goal is to develop an FHE compiler that not only vectorizes code but also reduces the instruction latency and noise growth while being more scalable.

RL for Code Optimization. RL has been explored for compiler code optimization [7, 11, 19, 29, 47, 58, 60, 63]. Systems such as *Halide RL* [47] train agents to select schedules for image processing code, while *Tiramisu RL* [7] applies polyhedral transformations to computational kernels using an RL-based policy. More specialized approaches include *NeuroVectorizer* [29], which learns to select the optimal vectorization and interleaving factors for loops on SIMD architectures using learned code embeddings, and *Polygym* [11], which frames affine loop transformation in the polyhedral model as a Markov Decision Process. Frameworks such as *CompilerGym* [19] have provided RL environments for tasks such as LLVM phase ordering. While previous work addresses traditional code optimizations, it does not consider the unique constraints of Fully Homomorphic Encryption. Our work is the first to formulate FHE optimization as an RL problem, where the agent must learn a policy that navigates the trade-offs between vectorization, cryptographic noise accumulation, and the cost of operations. Appendix G provides more details on using RL for code optimization.

9 Conclusion

This paper introduces a novel framework that leverages RL to automate FHE code optimization. Our proposed approach trains an RL agent to learn a policy for applying a sequence of rewriting rules to automatically vectorize scalar FHE code while reducing instruction latency and noise growth. We show that our approach generates code that is $5.3\times$ faster than Coyote, consumes $2.54\times$ less noise, while it takes $27.9\times$ less time in compiling code, enabling better scalability.

References

- [1] Andrew Adams, Karima Ma, Luke Anderson, Riyadh Baghdadi, Tzu-Mao Li, Michaël Gharbi, Benoit Steiner, Steven Johnson, Kayvon Fatahalian, Frédo Durand, and Jonathan Ragan-Kelley. 2019. Learning to optimize halide with tree

- search and random programs. *ACM Trans. Graph.* 38, 4, Article 121 (July 2019), 12 pages. doi:10.1145/3306346.3322967
- [2] Martin Albrecht, Melissa Chase, Hao Chen, Jintai Ding, Shafi Goldwasser, Sergey Gorbunov, Shai Halevi, Jeffrey Hoffstein, Kim Laine, Kristin Lauter, Satya Lokam, Daniele Micciancio, Dustin Moody, Travis Morrison, Amit Sahai, and Vinod Vaikuntanathan. 2019. Homomorphic Encryption Standard. *Cryptology ePrint Archive*, Paper 2019/939. <https://eprint.iacr.org/2019/939>
 - [3] Asra Ali, Jaeho Choi, Bryant Gipson, Shruthi Gorantala, Jeremy Kun, Wouter Legiest, Lawrence Lim, Alexander Viand, Meron Zerihun Demissie, and Hongren Zheng. 2025. HEIR: A Universal Compiler for Homomorphic Encryption. (August 2025). arXiv:2508.11095v1 [cs.CR] arXiv:2508.11095v1 [cs.CR].
 - [4] David W. Archer, José Manuel Calderón Trilla, Jason Dagit, Alex Malozemoff, Yuriy Polyakov, Kurt Rohloff, and Gerard Ryan. 2019. RAMPARTS: A Programmer-Friendly System for Building Homomorphic Encryption Applications. In *Proceedings of the 7th ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography* (London, United Kingdom) (WAHC '19). Association for Computing Machinery, New York, NY, USA, 57–68. doi:10.1145/3338469.3358945
 - [5] Frederik Armknecht, Colin Boyd, Christopher Carr, Kristian Gjøsteen, Angela Jäschke, Christian A Reuter, and Martin Strand. 2015. A guide to fully homomorphic encryption. *Cryptology ePrint Archive* (2015).
 - [6] Riyadh Baghdadi, Massinissa Merouani, Mohamed-Hicham Leghettas, Kamel Abdous, Taha Arbaoui, Karima Benatchba, and Saman Amarasinghe. 2021. A Deep Learning Based Cost Model for Automatic Code Optimization. arXiv:2104.04955 [cs.PL] <https://arxiv.org/abs/2104.04955>
 - [7] Riyadh Baghdadi, Jessica Ray, Malek Ben Romdhane, Emanuele Del Sozzo, Abdurrahman Akkas, Yunming Zhang, Patricia Suriana, Shoaib Kamil, and Saman Amarasinghe. 2019. Tiramisu: a polyhedral compiler for expressing fast and portable code. In *Proceedings of the 2019 IEEE/ACM International Symposium on Code Generation and Optimization* (Washington, DC, USA) (CGO 2019). IEEE Press, 193–205.
 - [8] Uday Bondhugula, Albert Hartono, J. Ramanujam, and P. Sadayappan. 2008. A practical automatic polyhedral parallelizer and locality optimizer. *SIGPLAN Not.* 43, 6 (June 2008), 101–113. doi:10.1145/1379022.1375595
 - [9] Zvika Brakerski. 2012. Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP. In *Advances in Cryptology – CRYPTO 2012*, Reihaneh Safavi-Naini and Ran Canetti (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 868–886.
 - [10] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. 2012. (Leveled) fully homomorphic encryption without bootstrapping. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference* (Cambridge, Massachusetts) (ITCS '12). Association for Computing Machinery, New York, NY, USA, 309–325. doi:10.1145/2090236.2090262
 - [11] Alexander Brauckmann, Andrés Goens, and Jeronimo Castrillon. 2021. PolyGym: Polyhedral Optimizations as an Environment for Reinforcement Learning. In *Proceedings of the 30th International Conference on Parallel Architectures and Compilation Techniques* (Atlanta, GA, USA) (PACT '21). IEEE Press, 17–29. doi:10.1109/PACT52795.2021.00009
 - [12] Alexander Brauckmann, Andrés Goens, and Jeronimo Castrillon. 2021. A reinforcement learning environment for polyhedral optimizations. *arXiv preprint arXiv:2104.13732* (2021).
 - [13] Yishen Chen, Charith Mendis, Michael Carbin, and Saman Amarasinghe. 2021. VeGen: a vectorizer generator for SIMD and beyond. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Virtual, USA) (ASPLOS '21). Association for Computing Machinery, New York, NY, USA, 902–914. doi:10.1145/3445814.3446692
 - [14] Eduardo Chielle, Oleg Mazonka, Homer Gamil, Nektarios Georgios Tsoutsos, and Michail Maniatakis. 2018. E3: A Framework for Compiling C++ Programs with Encrypted Operands. *Cryptology ePrint Archive*, Paper 2018/1013. <https://eprint.iacr.org/2018/1013>
 - [15] Sangeeta Chowdhary, Wei Dai, Kim Laine, and Olli Saarikivi. 2021. EVA Improved: Compiler and Extension Library for CKKS. *Cryptology ePrint Archive*, Paper 2021/1505. doi:10.1145/3474366.3486929 <https://eprint.iacr.org/2021/1505>.
 - [16] Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. 2022. Canine: Pre-training an Efficient Tokenization-Free Encoder for Language Representation. *Transactions of the Association for Computational Linguistics* 10 (2022), 73–91. doi:10.1162/tacl_a_00448
 - [17] Meghan Cowan, Deeksha Dangwal, Armin Alaghi, Caroline Trippel, Vincent T. Lee, and Brandon Reagan. 2021. Porcupine: A Synthesizing Compiler for Vectorized Homomorphic Encryption. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (Virtual, Canada) (PLDI 2021). Association for Computing Machinery, New York, NY, USA, 375–389. doi:10.1145/3453483.3454050
 - [18] Chris Cummins, Pavlos Petoumenos, Zheng Wang, and Hugh Leather. 2017. Synthesizing benchmarks for predictive modeling. In *2017 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 86–99. doi:10.1109/CGO.2017.7863731
 - [19] Chris Cummins, Bram Wasti, Jiadong Guo, Brandon Cui, Jason Ansel, Sahir Gomez, Somya Jain, Jia Liu, Olivier Teytaud, Benoit Steiner, Yuandong Tian, and Hugh Leather. 2022. CompilerGym: robust, performant compiler optimization environments for AI research. In *Proceedings of the 20th IEEE/ACM International Symposium on Code Generation and Optimization* (Virtual Event, Republic of Korea) (CGO '22). IEEE Press, 92–105. doi:10.1109/CGO53902.2022.9741258

- [20] Roshan Dathathri, Blagovesta Kostova, Olli Saarikivi, Wei Dai, Kim Laine, and Madan Musuvathi. 2020. EVA: An Encrypted Vector Arithmetic Language and Compiler for Efficient Homomorphic Computation. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (London, UK) (PLDI 2020). Association for Computing Machinery, New York, NY, USA, 546–561. doi:10.1145/3385412.3386023
- [21] Roshan Dathathri, Olli Saarikivi, Hao Chen, Kim Laine, Kristin Lauter, Saeed Maleki, Madanlal Musuvathi, and Todd Mytkowicz. 2019. CHET: An Optimizing Compiler for Fully-Homomorphic Neural-Network Inferencing. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Phoenix, AZ, USA) (PLDI 2019). Association for Computing Machinery, New York, NY, USA, 142–156. doi:10.1145/3314221.3314628
- [22] Fillipe DM de Souza, Hubert de Lassus, and Ro Cammarota. 2024. Private detection of relatives in forensic genomics using homomorphic encryption. *BMC Medical Genomics* 17, 1 (2024), 273.
- [23] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, Minneapolis, Minnesota, 4171–4186. doi:10.18653/v1/N19-1423
- [24] David Steven Dummit and Richard M Foote. 2004. *Abstract algebra*. Vol. 3. Wiley Hoboken.
- [25] Austin Ebel, Karthik Garimella, and Brandon Reagen. 2025. Orion: A Fully Homomorphic Encryption Framework for Deep Learning. *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '25)*, Article 16 (2025), 16 pages. arXiv:arXiv:2311.03470v3 [cs.CR] doi:10.1145/3676641.3716008
- [26] Alexandre E. Eichenberger, Peng Wu, and Kevin O'Brien. 2004. Vectorization for SIMD architectures with alignment constraints. *SIGPLAN Not.* 39, 6 (June 2004), 82–93. doi:10.1145/996893.996853
- [27] Junfeng Fan and Frederik Vercauteren. 2012. Somewhat Practical Fully Homomorphic Encryption. *Cryptology ePrint Archive*, Paper 2012/144. <https://eprint.iacr.org/2012/144>
- [28] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139
- [29] Ameer Haj-Ali, Nesreen K. Ahmed, Ted Willke, Yakun Sophia Shao, Krste Asanovic, and Ion Stoica. 2020. NeuroVectorizer: end-to-end vectorization with deep reinforcement learning. In *Proceedings of the 18th ACM/IEEE International Symposium on Code Generation and Optimization* (San Diego, CA, USA) (CGO '20). Association for Computing Machinery, New York, NY, USA, 242–255. doi:10.1145/3368826.3377928
- [30] Dong Huang, Guangtao Zeng, Jianbo Dai, Meng Luo, Han Weng, Yuhao Qing, Heming Cui, Zhijiang Guo, and Jie M. Zhang. 2025. EffiCoder: Enhancing Code Generation in Large Language Models through Efficiency-Aware Fine-tuning. arXiv:2410.10209 [cs.CL] <https://arxiv.org/abs/2410.10209>
- [31] Qijing Huang, Ameer Haj-Ali, William Moses, John Xiang, Ion Stoica, Krste Asanovic, and John Wawrzyniek. 2020. Autophase: Juggling hls phase orderings in random forests with deep reinforcement learning. *arXiv preprint arXiv:2003.00671* (2020).
- [32] Wang Huanting, Tang Zhanyong, Zhang Cheng, Zhao Jiaqi, Cummins Chris, Leather Hugh, and Wang Zheng. 2022. Automating Reinforcement Learning Architecture Design for Code Optimization. In *Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction* (Seoul, South Korea) (CC 2022). Association for Computing Machinery, New York, NY, USA, 129–143. doi:10.1145/3497776.3517769
- [33] Information Commissioner's Office. 2023. Case study: homomorphic encryption for data sharing. <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/data-sharing/privacy-enhancing-technologies/case-studies/homomorphic-encryption-for-data-sharing/> Accessed: 2025-08-17.
- [34] Lei Jiang and Lei Ju. 2022. FHEBench: Benchmarking Fully Homomorphic Encryption Schemes. arXiv:2203.00728 [cs.CR] <https://arxiv.org/abs/2203.00728>
- [35] T. Kamiya, S. Kusumoto, and K. Inoue. 2002. CCFinder: a multilingual token-based code clone detection system for large scale source code. *IEEE Transactions on Software Engineering* 28, 7 (2002), 654–670. doi:10.1109/TSE.2002.1019480
- [36] Ken Kennedy and John R. Allen. 2001. *Optimizing compilers for modern architectures: a dependence-based approach*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.
- [37] Miran Kim, Arif Ozgun Harmanci, Jean-Philippe Bossuat, Sergiu Carpov, Jung Hee Cheon, Ilaria Chillotti, Wonhee Cho, David Froelicher, Nicolas Gama, Mariya Georgieva, Seungwan Hong, Jean-Pierre Hubaux, Duhyeon Kim, Kristin Lauter, Yiping Ma, Lucila Ohno-Machado, Heidi Sofia, Yongha Son, Yongsoo Song, Juan Troncoso-Pastoriza, and Xiaoqian Jiang. 2021. Ultrafast homomorphic encryption models enable secure outsourcing of genotype imputation. *Cell Systems* 12, 11 (2021), 1108–1120.e4. doi:10.1016/j.cels.2021.07.010

- [38] Samuel Larsen and Saman Amarasinghe. 2000. Exploiting superword level parallelism with multimedia instruction sets. In *Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation* (Vancouver, British Columbia, Canada) (PLDI '00). Association for Computing Machinery, New York, NY, USA, 145–156. doi:10.1145/349299.349320
- [39] Kristin Lauter, Sreekanth Kannepalli, Kim Laine, and Radames Cruz Moreno. 2021. Password Monitor: Safeguarding passwords in Microsoft Edge. Microsoft Research Blog. <https://www.microsoft.com/en-us/research/blog/password-monitor-safeguarding-passwords-in-microsoft-edge/> Accessed: 2025-08-17.
- [40] Yongwoo Lee, Seonyeong Heo, Seonyoung Cheon, Shinnung Jeong, Changsu Kim, Eunkyung Kim, Dongyoon Lee, and Hanjun Kim. 2022. HECATE: Performance-Aware Scale Optimization for Homomorphic Encryption Compiler. In *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 193–204. doi:10.1109/CGO53902.2022.9741265
- [41] Long Li, Jianxin Lai, Peng Yuan, Tianxiang Sui, Yan Liu, Qing Zhu, Xiaojing Zhang, Linjie Xiao, Wenguang Chen, and Jingling Xue. 2025. ANT-ACE: An FHE Compiler Framework for Automating Neural Network Inference. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization (CGO '25)* (Las Vegas, NV, USA). Association for Computing Machinery, New York, NY, USA. doi:10.1145/3696443.3708924
- [42] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. 2013. On Ideal Lattices and Learning with Errors over Rings. *J. ACM* 60, 6, Article 43 (2013), 35 pages.
- [43] Raghav Malik, Kabir Sheth, and Milind Kulkarni. 2023. Coyote: A Compiler for Vectorizing Encrypted Arithmetic Circuits. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) (ASPLOS 2023). Association for Computing Machinery, New York, NY, USA, 118–133. doi:10.1145/3582016.3582057
- [44] Charith Mendis and Saman Amarasinghe. 2018. goSLP: globally optimized superword level parallelism framework. *Proc. ACM Program. Lang.* 2, OOPSLA, Article 110 (Oct. 2018), 28 pages. doi:10.1145/3276480
- [45] Massinissa Merouani, Khaled Afif Boudaoud, Iheb Nassim Aouadj, Nassim Tchoulak, Islem Kara Bernou, Hamza Benyamina, Fatima Benbouzid-Si Tayeb, Karima Benatchba, Hugh Leather, and Riyadh Baghdadi. 2025. LOOPer: A Learned Automatic Code Optimizer For Polyhedral Compilers. arXiv:2403.11522 [cs.PL] <https://arxiv.org/abs/2403.11522>
- [46] Julie L. Newcomb, Andrew Adams, Steven Johnson, Rastislav Bodik, and Shoaib Kamil. 2020. Verifying and Improving Halide’s Term Rewriting System with Program Synthesis. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 166 (nov 2020), 28 pages. doi:10.1145/3428234
- [47] Marcelo Pecenin, André Maidl, and Daniel Weingaertner. 2019. Optimization of Halide Image Processing Schedules with Reinforcement Learning. In *Proceedings of the 20th Symposium on High Performance Computing Systems* (Campo Grande). SBC, Porto Alegre, RS, Brasil, 37–48. doi:10.5753/wscad.2019.8655
- [48] Phitchaya Mangpo Phothilimthana, Archibald Samuel Elliott, An Wang, Abhinav Jangda, Bastian Hagedorn, Henrik Barthels, Samuel J. Kaufman, Vinod Grover, Emina Torlak, and Rastislav Bodik. 2019. Swizzle Inventor: Data Movement Synthesis for GPU Kernels. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) (ASPLOS '19). Association for Computing Machinery, New York, NY, USA, 65–78. doi:10.1145/3297858.3304059
- [49] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. 2021. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research* 22, 268 (2021), 1–8. <http://jmlr.org/papers/v22/20-1364.html>
- [50] Jonathan Ragan-Kelley, Andrew Adams, Dillon Sharlet, Connelly Barnes, Sylvain Paris, Marc Levoy, Saman Amarasinghe, and Frédo Durand. 2017. Halide: decoupling algorithms from schedules for high-performance image processing. *Commun. ACM* 61, 1 (Dec. 2017), 106–115. doi:10.1145/3150211
- [51] Gang Ren, Peng Wu, and David Padua. 2006. Optimizing data permutations for SIMD devices. *SIGPLAN Not.* 41, 6 (June 2006), 118–131. doi:10.1145/1133255.1133996
- [52] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [53] SEAL 2023. Microsoft SEAL (release 4.1). <https://github.com/Microsoft/SEAL>. Microsoft Research, Redmond, WA..
- [54] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Katrin Erk and Noah A. Smith (Eds.). Association for Computational Linguistics, Berlin, Germany, 1715–1725. doi:10.18653/v1/P16-1162
- [55] Alexander Shypula, Aman Madaan, Yimeng Zeng, Uri Alon, Jacob Gardner, Milad Hashemi, Graham Neubig, Parthasarathy Ranganathan, Osbert Bastani, and Amir Yazdanbakhsh. 2024. Learning Performance-Improving Code Edits. arXiv:2302.07867 [cs.SE] <https://arxiv.org/abs/2302.07867>
- [56] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction* (2nd ed.). The MIT Press.

- [57] Konrad Trifunovic, Dorit Nuzman, Albert Cohen, Ayal Zaks, and Ira Rosen. 2009. Polyhedral-Model Guided Loop-Nest Auto-Vectorization. In *2009 18th International Conference on Parallel Architectures and Compilation Techniques*. 327–337. doi:10.1109/PACT.2009.18
- [58] Mircea Trofin, Yundi Qian, Eugene Brevdo, Zinan Lin, Krzysztof Choromanski, and David Li. 2021. Mlgo: a machine learning guided compiler optimizations framework. *arXiv preprint arXiv:2101.04808* (2021).
- [59] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems 30*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.). Curran Associates, Inc. <http://papers.nips.cc/paper/7181-attention-is-all-you-need.pdf>
- [60] S. VenkataKeerthy, Siddharth Jain, Anilava Kundu, Rohit Aggarwal, Albert Cohen, and Ramakrishna Upadrasta. 2023. RL4ReAl: Reinforcement Learning for Register Allocation. In *Proceedings of the 32nd ACM SIGPLAN International Conference on Compiler Construction (CC '23)*. ACM, 133–144. doi:10.1145/3578360.3580273
- [61] Alexander Viand, Patrick Jattke, Miro Haller, and Anwar Hithnawi. 2022. HECO: Fully Homomorphic Encryption Compiler. In *USENIX Security Symposium*.
- [62] Alexander Viand, Patrick Jattke, and Anwar Hithnawi. 2021. SoK: Fully Homomorphic Encryption Compilers. In *2021 IEEE Symposium on Security and Privacy (SP)*. 1092–1108. doi:10.1109/SP40001.2021.00068
- [63] Zheng Wang and Michael O’Boyle. 2018. Machine Learning in Compiler Optimization. *Proc. IEEE PP* (05 2018), 1–23. doi:10.1109/JPROC.2018.2817118
- [64] Zhongcheng Zhang, Ying Liu, Yuyang Zhang, Zhenchuan Chen, Jiacheng Zhao, Xiaobing Feng, Huimin Cui, and Jingling Xue. 2025. Qiwu: Exploiting Ciphertext-Level SIMD Parallelism in Homomorphic Encryption Programs. In *Proceedings of the 2025 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, Las Vegas, Nevada, United States. Presented on Tue 4 Mar 2025, 15:20–15:40 at Casuarina Ballroom (Level 2), Security, Fault Tolerance & Cryptography Track.

A More Detailed Background

A.1 Fully Homomorphic Encryption

Fully Homomorphic Encryption (FHE) enables computations directly on encrypted data [5, 14]. A *homomorphism* is a function between two groups that preserves their structure [24]. In the context of FHE, these groups correspond to elements in the plaintext space $\mathcal{P}$ and the ciphertext space $\mathcal{C}$. Let us define the encryption and decryption functions as $E(\cdot) : \mathcal{P} \mapsto \mathcal{C}$ and $D(\cdot) : \mathcal{C} \mapsto \mathcal{P}$, respectively. Given an operation $\circ$ on plaintexts and its homomorphic counterpart $\odot$ on ciphertexts, for plaintexts $m_a, m_b \in \mathcal{P}$ with encryptions $c_a = E(m_a), c_b = E(m_b) \in \mathcal{C}$, the homomorphic property ensures that operations on encrypted data remain consistent with their plaintext counterparts, i.e., $m_a \circ m_b = D(c_a \odot c_b)$.

A.1.1 BFV Scheme. The BFV encryption scheme [27] is an FHE scheme based on the Ring-Learning With Errors (RLWE) problem [42]. In BFV, the plaintext space is defined as $\mathcal{P} = R_t = \mathbb{Z}_t[x]/(x^n + 1)$, while the ciphertext space is $\mathcal{C} = R_q \times R_q$, where $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. Here, n is the polynomial modulus degree, and t, q , and $x^n + 1$ denote the plaintext, ciphertext, and polynomial moduli. Typical values for t range from 16 to 32 bits, while q varies between 100 and 900 bits. The degree n is usually a power of two, commonly between 2^{10} and 2^{16} , as recommended by the *Homomorphic Encryption Standard* [2].

A.1.2 FHE Batching. Batching enables evaluation of a function on n blocks of data [10] at once by encoding a vector of n messages in a single plaintext polynomial in R_t . Under the assumption $t \equiv 1 \pmod{2n}$, $x^n + 1$ factors into linear polynomials modulo t , i.e. $x^n + 1 \equiv \prod_{i=1}^n (x - a_i) \pmod{t}$, with $a_i = a^{2^{i-1}}$ being the $2n^{\text{th}}$ primitive root of unity modulo t , and $t = \prod_{i=1}^n t_i$ where t_i are prime ideals with basis $(t, x - a_i)$. Using CRT on ideals, we have the following isomorphism:

$$R_t = R/(t) \stackrel{\text{CRT}}{\cong} R/t_1 \times \cdots \times R/t_n$$

Thus, evaluating a function once over $R/(t)$ evaluates the same function on smaller plaintext spaces $R/t_1, \dots, R/t_n$.

B CHEHAB Compiler

This section provides an overview of the CHEHAB compiler and its components. We first show the overall design of the CHEHAB compiler and then discuss its major components, including our RL agent for code optimization. CHEHAB defines a domain-specific language (DSL) for FHE. The CHEHAB DSL is the compiler’s starting point. An intermediate representation (IR) is first generated from the CHEHAB DSL. This IR is then optimized by a Reinforcement learning (RL) agent, which learns a policy to apply a sequence of transformations. These transformations are drawn from a set of rewriting rules that include rules for both code vectorization and for simplification to reduce noise growth and instruction latency (i.e., rewrite complex arithmetic expressions into simpler expressions that have a smaller number of operations and depth). Finally, the code generator generates a sequence of vectorized operations implemented in C++ and targeting Microsoft SEAL’s backend [53] for BFV. Our main contribution in this paper is the introduction of a Term Rewriting System that uses constrained reinforcement learning to select the rewriting rules to apply. We call this the Constrained RL TRS.

B.1 CHEHAB Domain Specific Language (DSL)

FHE performance and correctness critically depend on managing the encryption parameters and rotation keys. Managing these keys and parameters manually is a time-consuming task that requires expertise. The CHEHAB DSL is designed to manage this complexity automatically, and thus improves the productivity of the developer. The developer expresses their code using a simple, high-level C++ code, while CHEHAB takes care of lowering that code to the FHE paradigm and takes care of generating and managing all the FHE-related objects (such as the encryption keys). CHEHAB is embedded in C++, allowing users to leverage C++ features (e.g., templates) while extending it by defining the Plaintext and Ciphertext types. It also overloads basic operations for these types, mapping them to native FHE operations. One of the guiding principles in designing CHEHAB is the clear separation between computations and implementation details. In particular, it allows computations to be specified independently from encryption parameters, offering greater flexibility in exploiting potential optimization opportunities.

A program written in CHEHAB has three parts: (1) Input declaration: This section defines the inputs of the program. An input can be either a ciphertext or a plaintext. It can also be either a scalar or a vector. (2) Computations: This section specifies the computations, whether arithmetic or rotation operations, using standard C++ operators (+, -, *, «). These operators are mapped to either vector or scalar operators depending on the type of their operands. (3) Outputs: This section declares the outputs.

The following example shows how to express the motivating example in our DSL. The list of operations supported by CHEHAB is presented in Table 2 in the appendix.

```
Ciphertext motivating_example() {
    Ciphertext v1,v2,v3,v4,v5,v6,v7,v8,v9,v10; //Inputs
    Ciphertext x = (((v1 * v2) * (v3 * v4)) + ((v3 * v4) *
(v5 * v6))) * ((v7 * v8) * (v9 * v10)); //Computations
    x.set_output(); //Outputs
    return x;
}
```

B.2 List of Operations in the CHEHAB DSL

Table 2 provides the full list of operations in the CHEHAB DSL as well as their signatures and descriptions.

Table 2. Operations of the CHEHAB domain-specific language (ct: ciphertext, pt: plaintext, int: integer).

Op.	Signature	Description
+	$ct \times ct \rightarrow ct$	Element-wise addition
	$ct \times pt \rightarrow ct$	
	$pt \times ct \rightarrow ct$	
+	$ct \times int \rightarrow ct$	Add int value to all elements of ct
-	$ct \times ct \rightarrow ct$	Element-wise subtraction
	$ct \times pt \rightarrow ct$	
	$pt \times ct \rightarrow ct$	
-	$ct \times int \rightarrow ct$	Subtract int value from all elements of ct
-	$ct \rightarrow ct$	Negation of each element of the argument
<<	$ct \times int \rightarrow ct$	Rotation of ciphertext to the left with the given step
>>	$ct \times int \rightarrow ct$	Rotation of the ciphertext to the right with the given step
*	$ct \times ct \rightarrow ct$	Element-wise multiplication
	$ct \times pt \rightarrow ct$	
	$pt \times ct \rightarrow ct$	
*	$ct \times int \rightarrow ct$	Multiply all ciphertext elements by an int

B.3 CHEHAB Intermediate Representation (IR)

The DSL program is compiled into an intermediate representation (IR) in the form of an AST (Abstract Syntax Tree), as is commonly done in compilers. Fig. 1a shows an example of an AST representation of the motivating example. From this representation, we extract a dataflow graph representation (DAG) of the code when needed (e.g., to calculate the multiplicative depth). To lower the DSL to the IR, we follow the same approach of Halide [50] and Tiramisu [7]. The approach relies on the use of templates and staging in C++.

B.4 Code Optimizations

After generating the IR, the next stage is code optimization. CHEHAB RL uses an RL agent to optimize code. The agent chooses a sequence of rules from its rule set. This set includes rules for *vectorization*, which group scalar operations into vector ones, and rules for *simplification*, which minimize noise growth and latency in the circuit. To further optimize code, the original CHEHAB compiler applies classic compiler optimizations, including common subexpression elimination (CSE), constant folding, and dead code elimination.

B.5 Code generation

In the final stage, CHEHAB generates code that targets the SEAL API [53] by mapping each node in the IR (operator) to its equivalent API call in SEAL. During code generation, we also generate the necessary masking operations, private keys, public keys, and relinearization keys.

To reduce memory consumption, CHEHAB minimizes the creation of temporary ciphertext and plaintext objects using the primitive *inplace* provided by the SEAL API. This primitive enforces the reuse of memory occupied by dead objects, similar to how a compound assignment operator

works. To compile a program, the user runs the DSL code. When the DSL code is run, it creates the IR AST, runs the compiler passes on the IR, and generates optimized code at the end (C++ code). This code is further compiled using a C++ compiler. The generated binary code is the final outcome of the compilation process and can be run like any other binary. This approach is common in implementing DSLs embedded in C++.

C Rewriting Rules

The transformations available to the RL agent are defined as a set of rewriting rules. These rules include rules for vectorization, algebraic simplification, and rotation.

Vectorization Rules for Isomorphic Subexpressions. These rules pack scalar arithmetic operations into single vector instructions. They search for isomorphic element-wise scalar expressions and rewrite them as a single vector instruction. For example, a vectorization rule for addition is:

$$(\text{Vec } (+ \ a \ b) \ (+ \ c \ d)) \Rightarrow (\text{VecAdd } (\text{Vec } a \ c) \ (\text{Vec } b \ d))$$

This rewriting rule replaces two scalar additions (left-hand side) with one vector addition operating on newly constructed vectors of the operands (right-hand side). Similar rules exist for multiplication, subtraction, and negation.

Vectorization Rules for Non-isomorphic Subexpressions. While powerful, the previous vectorization rules require isomorphic subexpressions. To handle common patterns of non-isomorphic subexpressions, our set of rules also includes general rules for vectorizing such non-isomorphic patterns. For each arithmetic operation op , we have a rule that matches whenever an expression contains a mix of operations (non-isomorphic subexpressions), provided that the operation op appears more than once. The rule then vectorizes all instances of the operation op . It also moves any non-matching sub-expression (containing operators other than op) into the first operand vector. It then pads the second operand vector with the appropriate identity element (e.g., 1 for multiplication, 0 for addition). For example:

$$(\text{Vec } (* \ a \ b) \ (* \ c \ d) \ (- \ f \ g)) \Rightarrow \\ (\text{VecMul } (\text{Vec } a \ c \ (- \ f \ g)) \ (\text{Vec } b \ d \ 1))$$

Here, the non-isomorphic vectorization rule for multiplication matches because there are two $*$ operations. It packs the two multiplications into a single vector multiplication operation, and leaves $(- \ f \ g)$ in the 1st operand, and pads the second operand vector with 1, the multiplicative identity.

Since the RL agent is trained to predict rewriting rules that maximize the global reward, it automatically weights the cost and benefit of applying this type of vectorization and selects this class of rules only if they are beneficial.

Simplification and Algebraic Rules: This category includes standard algebraic rules that simplify expressions, reduce computational complexity, or transform expressions into a different form (that will be useful in later simplifications). These rules aim to lower the circuit’s depth and minimize the number of operations. Examples include:

- *Arithmetic Simplification:* These rules simplify arithmetic expressions, replacing complex or multiple operations with simpler, equivalent ones. Examples of these rules include factorization

$$(+ \ (* \ x \ y) \ (* \ x \ z)) \Rightarrow (* \ x \ (+ \ y \ z)),$$

identity elimination

$$(x \ * \ 1) \Rightarrow x,$$

absorption rules

$$(x \ * \ 0) \Rightarrow 0,$$

plaintext consolidation

$$(* (pt\ a) (* (pt\ b)\ x)) \Rightarrow (* (pt\ (a*b))\ x),$$

where a and b are plaintexts.

- *Arithmetic Transformations*: These rules transform arithmetic expressions in a way that enables their simplification later. Examples include commutativity, associativity, and distribution rules.
- *Circuit Balancing*: These rules balance expression trees, reducing their depth (and noise accumulation). For example, a left-leaning tree of multiplications can be balanced to reduce the multiplicative depth:

$$(\text{VecMul } x (\text{VecMul } y (\text{VecMul } z\ t))) \Rightarrow (\text{VecMul } (\text{VecMul } x\ y) (\text{VecMul } z\ t))$$

Rotation Rules: Data alignment is critical in FHE, and rotations are the primary mechanism for moving data within a packed ciphertext. A naive approach to vectorizing unstructured code would require the RL agent to discover a long and potentially inefficient sequence of low-level rotation, masking, and arithmetic operations to correctly align data. To address this, our set includes rules that transform high-level computational patterns directly into efficient, *composite dataflow structures*. These rules encapsulate what would otherwise be multiple low-level steps into a single, high-level transformation. For example, we consider this expression:

$$(\text{Vec } ((+ (* a\ b) (* c\ d)) (+ (* e\ f) (* g\ h))))$$

A standard vectorization strategy, without leveraging rotations, would result in the following structure:

$$(\text{VecAdd } (\text{VecMul } (\text{Vec } a\ e) (\text{Vec } b\ f)) (\text{VecMul } (\text{Vec } c\ g) (\text{Vec } d\ h)))$$

This optimized expression requires two vector multiplication operations and one vector addition operation. However, a more sophisticated strategy, enabled by our rotation-based rules, can find a more efficient solution:

$$(\text{VecAdd } V (<< V\ 2))$$

where $V: (\text{VecMul } (\text{Vec } a\ c\ e\ g) (\text{Vec } b\ d\ f\ h))$

The final result can then be computed by adding this vector V to a rotated version of itself, which effectively sums the required pairs of products into the first two slots. This alternative strategy requires only one vector multiplication operation, one vector addition, and one rotation. Since a vector rotation is cheaper than a vector multiplication in FHE, this second approach is better. By including such composite transformation rules, they help the RL agent to discover these globally optimal strategies that a simpler vectorizer would miss.

C.1 How did we design our rewriting rules?

To construct the rules for the TRS, we began by collecting the rules from Halide’s TRS [46, 50]. Halide is an industrial compiler used for the optimization of image processing and deep learning pipelines. Halide’s expression space includes operations that are not natively supported by fully homomorphic encryption (FHE), such as comparison, division, and modulo operations. Additionally, performance considerations and optimization goals in Halide differ from ours, as it operates on plaintext data. Therefore, we selected only the rules that are compatible with FHE. Next, we expanded this initial ruleset manually and developed new rules aimed at reducing the number of operations, rotations, depth, and multiplicative depth in FHE programs.

D Identifier and Constant Invariant (ICI) Tokenization

The goal of this tokenization method is to create a canonical representation for a given program, P . Different programs that are semantically equivalent but differ only in the variable names and constants will have the same tokenization. The result is a canonical token sequence, $T(P)$, that is invariant to the specific names of variables and constants.

This method aims to extract the program semantics regardless of the variable names or constant values used in the program. It is designed to be simple and fast. It uses a small, fixed vocabulary for all operators (e.g., $+$, $*$, VecAdd) and for parentheses ($(\dots)$). For each program P being tokenized, a mapping is created for its variables and constants. The first unique variable encountered is mapped (renamed) to a generic ID, v_0 , the second to v_1 , and so on. A similar mapping is applied to numerical constants (mapped to c_0 , c_1 , etc.), with an important exception: the integers **0** and **1** are kept as part of the fixed vocabulary, as they have special semantic meaning in many rewrite rules (e.g., additive and multiplicative identities). All other constants (e.g., 5, 42) are renamed in a way similar to variables (yet, we preserve the information that they are numerical constants). The reason for applying ICI tokenization to numerical constants is that programs that are semantically equivalent but only differ in the constant values are usually optimized in the same way, and therefore, there is no need to preserve the actual values of the constants. We also do not preserve the actual values of constants since our rewriting rules do not use such information. All we need to preserve is whether the numerical constants used in different positions within a given expression are the same. This allows the system to recognize that two programs that differ only in variable names and values of constants are, for optimization purposes, semantically equivalent. For example, $(+ a (* b c))$ and $(+ x (* y z))$ are both mapped to the same canonical token sequence $(+ v_0 (* v_1 v_2))$. The expressions $(+ (+ a b) (+ 5 6) (+ a 5))$ and $(+ (+ c d) (+ 10 8) (+ c 10))$ are both mapped to the same canonical token sequence $(+ (+ v_0 v_1) (+ c_0 c_1) (+ v_0 c_0))$. Without this method, the model would treat ‘a’ and ‘x’, in the first example, as entirely unrelated tokens, and it would need to see countless examples to learn that the underlying computational pattern is the same. This invariance enables the encoder to learn the same representation for programs that are semantically equivalent but only differ in their variable names or constant values. Therefore, ICI tokenization transforms a difficult learning problem into a much simpler one.

Identifier and Constant Invariant (ICI) tokenization is faster because it just scans the program once and assigns each new name a short canonical ID, while Byte-Pair Encoding (BPE) tokenization [54] repeatedly checks a large list of merge rules to decide how to split and join characters. More precisely, ICI tokenization uses a small fixed vocabulary for operators and delimiters and per-program hash maps for variables/constants, so tokenization is a single left-to-right pass with $O(1)$ updates per new symbol; BPE, in contrast, uses a large learned inventory of subword pieces. It tokenizes by repeatedly finding the longest piece that matches the next characters (i.e., searching a big dictionary for the longest prefix) and then applying learned merge rules until no more merges apply. These repeated searches and merges over a large vocabulary add overhead.

E Training Details

The agent was trained on the LLM-generated dataset. We used a single node with a 2.40 GHz Intel(R) Xeon(R) CPU E5-2680 v4 (14 cores) and 256 GB DDR4 of memory running CentOS Linux 8 for the training. We used the *Stable-Baselines3 library* [49] to implement our RL training. To accelerate the collection of experience, we leverage parallel environments. We run multiple independent instances of the optimization environment in parallel (8 in our case), allowing the agent to collect a diverse batch of trajectories simultaneously. To implement this, we leverage multi-processing

where each environment runs in its own process, and multiple episodes are collected in parallel across processes. This speeds up the training process.

Each training episode corresponds to the optimization of a single expression and is limited to a maximum length of 75 steps. We found this limit to be sufficient, as the optimization sequences for the majority of expressions in our training and evaluation sets finished well before reaching 75 steps. The agent was trained for a total of 2 million timesteps, and the training took 43 hours.

Table 3. PPO Hyperparameters for Training.

Hyperparameter	Value
Learning Rate	1×10^{-4}
Discount Factor (γ)	0.99
GAE Lambda (λ)	0.95
PPO Clip Range (ϵ)	0.2
Update Epochs	20
Steps per Update	2048
Batch Size	256
Number of Environments	8

F More Details about the Evaluation

F.1 Noise Measurement

A freshly encrypted ciphertext begins with a large noise budget, and each homomorphic operation consumes a part of it. When the budget reaches zero, decryption fails. Our goal is to minimize the noise consumed by code.

To measure the noise consumed in each benchmark, we use the function `Decryptor::invariant_noise_budget` provided by the Microsoft SEAL library. In the BFV scheme, every ciphertext carries an error term (noise) that grows as homomorphic operations are performed. SEAL reports the remaining noise budget (in bits).

In our setup, using a polynomial modulus degree of 16384, a freshly encrypted ciphertext has an initial noise budget of 369 bits. After running each benchmark, we query the remaining noise budget and measure the difference from the initial value. The difference is the consumed noise that we report (in bits).

We use a 20-bit plaintext modulus and select the coefficient modulus using SEAL’s helper

```
CoeffModulus::BFVDefault(poly_modulus_degree)
```

This returns a coefficient modulus with a total size of 389 bits (`total_coeff_modulus_bits = 389`), which is consistent with a theoretical maximum initial noise budget of $389 - 20 = 369$ bits (i.e., `total_coeff_modulus_bits - plain_modulus_bits`).

F.2 Random Code Generation

Our random code generator recursively constructs IR expressions. We control this construction by two parameters: maximum depth and vector size. The generator builds expressions by sampling a mixture of scalar operations, vector operations, rotations, and vector constructors. Sampling is balanced across all combinations of depth (1–15) and vector size (1–32), so the model encounters a wide range of shapes and packing patterns.

Concretely, the generator works as follows:

- (1) It samples a target vector size n and a depth budget d .

Table 4. Notation.

Symbol	Description
$\cup$	depth including all operations
$\cup^{\otimes}$	multiplicative depth
$\oplus$	# of ciphertext additions
$\otimes$	# of ciphertext-ciphertext multiplications
$\odot$	# of ciphertext-plaintext multiplications
$\boxtimes$	# of ciphertext squares
$\cup$	# of rotations

- (2) Starting from a root, it grows an expression tree until the tree depth budget is reached.
- (3) At each node, it picks an operator from $\{\text{scalar op}, \text{vector op}, \text{Vec constructor}\}$ that is type-compatible and respects the chosen vector size n .
- (4) Leaves are instantiated with variables or integer constants.
- (5) The resulting expression is parsed and type-checked into the compiler IR; invalid samples are discarded.

F.3 Randomly Generated Unstructured Kernels

We borrow the following description of the randomly generated irregular polynomials from the original Coyote [43] paper, which proposed the benchmark. Several polynomials are randomly generated to evaluate as arbitrary arithmetic expression trees. The trees are generated according to three different regimes to cover different kinds of programs:

- Dense, homogeneous: The expression tree is both full and complete, and all the operations are isomorphic. In principle, this represents a best case for vectorization. We refer to these as tree-100-100.
- Dense, nonhomogeneous: The expression tree is both full and complete, each operation has a 50/50 chance of being an add or a multiply. Hence, while the trees are structurally similar, the heterogeneity of operations means that vectorization opportunities are restricted. We refer to these as tree-100-50.
- Sparse: Many operations have one leaf node input; the tree is not very balanced. In principle, this represents a worst case for vectorization, where Coyote must work hard to find vectorizable computation. We refer to these as tree-00-50.

The last number in the names of the polynomials indicates the depth of the tree. For example, tree-100-100-5 indicates a dense, homogeneous tree with a depth equal to 5.

F.4 Benchmark Evaluation Results

Table 5 presents several metrics regarding the depth and number of operations for three cases: 1) the initial (naive) implementation of the benchmarks 2) CHEHAB RL’s generated code for each one of the benchmarks, and 3) Coyote’s generated code. The table compares the three cases in terms of circuit depth ($\cup$), multiplicative depth ($\cup^{\otimes}$), number of ciphertext-ciphertext multiplications ($\otimes$), number of rotations ($\cup$), number of ciphertext-plaintext multiplications ($\odot$), and number of ciphertext additions ($\oplus$). We also report CHEHAB’s and Coyote’s compilation times CT (s) and their Consumed Noise (CN), where lower values are better. The notation used in Table 5 is presented in Table 4. The most important columns in the table are the multiplicative depth ($\cup^{\otimes}$), the number of rotations ($\cup$), the compilation time (CT) for the two compilers, and the Consumed Noise (CN). We highlight them in bold.

Table 5. Comparison between an initial (naive) implementation, CHEHAB, and code generated by Coyote, in terms of circuit depth ($\cup$) and multiplicative depth ($\cup^\otimes$), number of ciphertext-ciphertext multiplications ($\otimes$), number of rotations ($\cup$), number of ciphertext-plaintext multiplications ($\odot$), and number of ciphertext additions ($\oplus$). We also report CHEHAB’s and Coyote’s compilation times CT (s) and their Consumed Noise (CN). The important columns in the Table are in bold.

Kernel	Initial						CHEHAB RL						CN	CT	Coyote						CN	CT
	$\cup$	$\cup^\otimes$	$\otimes$	$\cup$	$\odot$	$\oplus$	$\cup$	$\cup^\otimes$	$\otimes$	$\cup$	$\odot$	$\oplus$			$\cup$	$\cup^\otimes$	$\otimes$	$\cup$	$\odot$	$\oplus$		
Box Blur 3×3	9	0	0	0	0	31	6	0	0	1	0	6	13	9.570	25	0	0	18	45	24	141	471.991
Box Blur 4×4	9	0	0	0	0	74	5	0	0	1	0	5	12	10.146	36	0	0	51	111	26	247	926.780
Box Blur 5×5	9	0	0	0	0	135	4	0	0	0	0	8	10	10.457	36	0	0	100	204	27	250	1553.897
Dot Product 4	5	1	4	0	0	4	6	1	1	3	0	2	45	9.826	7	1	1	2	4	4	42	106.571
Dot Product 8	9	1	8	0	0	8	8	1	1	4	0	3	47	9.864	9	1	1	3	4	6	93	179.196
Dot Product 16	17	1	16	0	0	16	10	1	1	5	0	4	47	11.068	15	1	1	10	14	11	119	324.983
Dot Product 32	33	1	32	0	0	32	11	1	1	5	0	5	44	20.492	22	1	1	21	34	16	145	629.485
Hamm. Dist. 4	7	2	8	0	4	8	8	2	2	4	0	3	74	9.421	3	2	2	0	0	1	99	144.852
Hamm. Dist. 8	11	2	16	0	8	16	10	2	2	5	0	4	74	11.416	4	2	2	3	0	1	73	282.906
Hamm. Dist. 16	19	2	32	0	16	32	12	2	2	6	0	5	75	29.736	12	2	2	21	42	12	233	520.420
Hamm. Dist. 32	35	2	64	0	32	64	14	2	3	5	0	7	76	178.241	12	2	2	69	92	8	156	1054.597
L2 Distance 4	5	1	4	0	0	3	8	1	1	4	0	2	45	9.445	8	1	1	2	4	4	68	158.341
L2 Distance 8	9	1	8	0	0	7	9	1	1	4	0	3	45	10.350	11	1	2	3	9	9	122	276.027
L2 Distance 16	17	1	16	0	0	15	11	1	1	5	0	4	46	20.696	22	1	2	23	52	18	176	574.150
L2 Distance 32	33	1	32	0	0	31	12	1	1	5	0	5	45	95.935	26	1	1	73	101	17	179	1170.184
Linear Reg. 4	3	1	4	0	4	8	5	1	1	2	0	2	41	9.360	3	1	1	0	0	0	41	2.278
Linear Reg. 8	3	1	8	0	8	16	5	1	1	2	0	2	41	9.469	6	1	1	1	6	3	66	197.718
Linear Reg. 16	3	1	16	0	16	32	4	1	1	1	0	2	41	9.547	12	1	1	17	25	7	122	500.449
Linear Reg. 32	3	1	32	0	32	64	3	1	1	0	1	2	10	9.778	12	1	1	44	46	4	124	963.687
Poly. Reg. 4	5	2	12	0	0	12	8	2	2	3	0	3	74	10.139	16	2	3	4	19	10	153	238.869
Poly. Reg. 8	5	2	24	0	0	24	7	2	2	2	0	3	73	9.786	20	2	2	21	38	12	205	501.163
Poly. Reg. 16	5	2	48	0	0	48	6	2	2	1	0	3	73	9.993	20	2	2	56	79	12	208	961.519
Poly. Reg. 32	5	2	96	0	0	96	5	2	2	0	0	3	73	10.145	20	2	2	134	173	12	210	1896.952
Gx 3×3	4	1	24	0	24	33	8	2	4	1	2	6	66	9.463	16	1	1	27	57	11	150	594.926
Gx 4×4	4	1	48	0	48	64	6	1	3	1	1	4	42	10.743	16	1	1	59	112	11	150	1061.439
Gx 5×5	4	1	80	0	80	100	5	1	3	0	2	5	40	10.646	16	1	1	105	182	12	154	1673.958
Gy 3×3	4	1	24	0	24	33	6	1	3	1	1	3	41	9.680	16	1	1	29	54	10	149	595.976
Gy 4×4	4	1	48	0	48	64	6	1	3	1	0	3	42	10.591	16	1	1	60	101	11	197	1057.486
Gy 5×5	4	1	80	0	80	105	6	1	5	0	0	6	42	11.322	17	1	1	103	169	11	154	1670.887
Rob. Cross 3×3	6	2	42	0	24	63	5	1	2	2	0	3	45	9.786	12	1	2	10	27	11	96	291.833
Rob. Cross 4×4	6	2	76	0	44	112	5	1	2	2	0	3	44	10.363	12	1	2	24	51	9	125	543.704
Rob. Cross 5×5	6	2	120	0	70	175	4	1	2	0	0	4	42	11.681	12	1	2	43	95	10	126	857.648
Mat. Mul. 3×3	3	1	27	0	0	18	4	1	2	1	0	2	41	9.426	9	1	2	9	14	5	93	456.644
Mat. Mul. 4×4	4	1	64	0	0	48	4	1	2	1	0	2	42	10.039	10	1	2	32	44	7	94	1085.128
Mat. Mul. 5×5	5	1	125	0	0	100	4	1	5	0	0	4	42	11.601	14	1	1	69	82	13	145	2158.145
Max 3	6	2	6	0	0	6	9	2	3	3	0	4	76	9.481	14	2	3	3	10	9	75	132.890
Max 4	8	3	12	0	0	12	11	3	7	3	0	8	107	11.784	22	3	6	8	25	16	235	252.436
Max 5	10	4	20	0	0	20	15	4	12	5	0	12	140	149.936	29	4	9	27	55	21	321	509.189
Sort 3	8	3	10	0	0	8	11	3	5	3	0	6	107	10.552	12	3	4	5	6	9	188	184.944
Sort 4	14	6	58	0	0	35	21	6	26	5	0	35	206	522.043	42	6	10	66	114	31	369	959.704
Tree 50-50-5	4	2	2	0	0	4	8	2	2	3	0	3	73	9.271	4	2	2	0	0	4	73	4.306
Tree 50-50-10	10	5	16	0	0	16	15	5	15	2	0	18	172	18.170	27	7	7	7	19	21	243	147.033
Tree 100-50-5	5	3	15	0	0	16	13	5	8	3	0	13	169	12.428	18	3	4	7	22	14	212	176.904
Tree 100-50-10	10	8	519	0	0	504	16	8	509	2	0	511	334	148.398	50	8	148	490	873	243	369	6798.692
Tree 100-100-5	5	4	29	0	0	2	10	4	4	5	0	1	141	11.152	16	5	5	5	13	7	211	170.154
Tree 100-100-10	10	9	1022	0	0	1	12	9	256	2	0	1	307	91.627	36	10	10	334	373	16	369	5396.742

F.5 GRU vs. Transformer Encoder

To validate our choice of a Transformer encoder for the state representation, we conducted an experiment to compare its encoding capabilities against a baseline recurrent architecture, the Gated Recurrent Unit (GRU). For this comparison, we constructed complete autoencoders for both architectures. The quality of the reconstruction serves as a direct measure of the quality of embedding learned by the encoder; a perfect reconstruction implies that the encoder has preserved all necessary structural information of the program IR. Our RL agent uses only the encoder

component, but this autoencoder setup allows us to empirically verify its ability to learn an embedding for the input program.

Experimental Setup. Both autoencoders were trained on the same dataset of 1.4 million randomly generated IR expressions. We use the same methodology described in Sec. F.2 to generate this dataset. The GRU baseline used a 4-layer bidirectional encoder and a 4-layer decoder. The Transformer autoencoder also used 4 encoder and 4 decoder layers, with both models using identical optimizer settings and batch sizes.

Results. The training curves in Fig. 7 and the final test results in Table 6 show a clear distinction in performance. The Transformer autoencoder not only learns significantly faster but also achieves 100% exact-match reconstruction accuracy on the test set. The GRU model, however, plateaus at 98.92% exact-match accuracy.

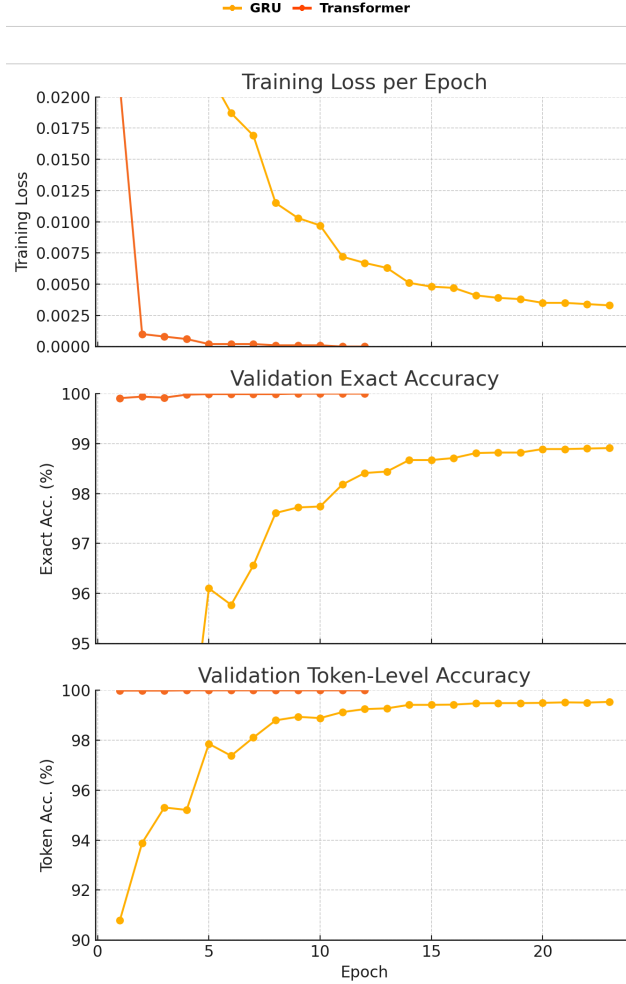


Fig. 7. Validation loss and accuracy comparison between the Transformer and GRU-based autoencoders. RNN-AE in the figure represents the GRU-based auto-encoder.

Table 6. Autoencoder reconstruction accuracy on the validation and test set.

Model	Validation		Test	
	Exact (%)	Token (%)	Exact (%)	Token (%)
GRU Autoencoder	98.91	99.54	98.92	99.52
Transformer Autoenc.	100.00	100.00	100.00	100.00

Analysis of Results. The primary source of failure for the GRU was ordering errors, where it produced the correct set of tokens but in an incorrect order. A detailed analysis showed that 9.8% of the GRU’s reconstructions had a non-zero edit distance from the ground truth, typically due to misplaced parentheses or swapped sibling nodes in the expression tree. This confirms the hypothesis that the GRU’s sequential processing and fixed-length hidden state are insufficient to reliably preserve the long-range structural information of complex IRs. The Transformer’s self-attention mechanism, in contrast, processes all tokens in parallel, allowing it to directly model these dependencies and achieve lossless reconstruction. These results validate our choice of the Transformer encoder for state representation.

G More Detailed Related Work

FHE Compilers for Circuit Optimization. EVA [20] is an example of such a compiler, it focuses on automatic parameters selection and ciphertext maintenance to manage noise and scale of ciphertexts for the CKKS scheme, however, it does not apply any optimization strategies in order to reduce the multiplicative depth or the number of operations. In a later version of the same work [15], CSE (Common Subexpression Elimination) was added, which provides basic simplifications of the circuit (in CSE, redundant expressions are computed once and used multiple times). CHEHAB also applies CSE on the generated code, but in addition to that, it applies term rewriting to reduce the number of operations and the multiplicative depth. Ramparts [4] uses Julia as input language and targets the BFV scheme. It provides automatic parameter selection based on noise growth estimation, it also simplifies circuits in order to reduce the number of operations by applying simple and classical optimization techniques such as CSE and constant folding, it also offers optimizations such as loop unrolling, and function inlining. It does insert ciphertext-maintenance operation, but in a naive way [62]. Unlike CHEHAB, Ramparts cannot optimize vectorized code [62], which affects its ability to take advantage of SIMD parallelism and improve latency. HECATE [40] is a recent work that focuses on selecting the parameter q for CKKS, and at the same time it performs ciphertext-maintenance operations scheduling by applying rewrite on the input circuit, HECATE outperforms EVA by 27.38%, however, it does not apply any optimization to reduce the number of operations and multiplicative depth of the circuit.

RL-based methods for Code Optimization. Recent attempts explored the use of reinforcement learning to solve the problem of choosing the right sequence of code transformations. In PolyGym [12] and CompilerGym [19], the authors propose only RL environments without implementing RL agents to optimize code; their main contribution is to show that their action space has potentially good optimizations to explore. They leave the implementation of an RL agent as future work.

Other work such as HalideRL [47], AutoPhase [31] and SuperSonic [32] propose RL agents to optimize code. HalideRL is not fully automatic. The user has to provide an initial set of code transformations. The HalideRL agent then discards transformations that are not useful and keeps only those that are useful. It then selects the best parameters for the useful transformations. In addition, HalideRL does not generalize to programs unseen during training. It is trained on a given program with multiple random data input sizes. Then, during deployment, it is used to

optimize that same program. This is different from our approach. Our RL agent is designed to generalize to programs unseen during training. We first train our RL agent on a large set of random LLM-generated programs. Once it learns how to optimize them, we then deploy it on new unseen programs and use it to optimize them.

SuperSonic [32] is a meta-optimizer that targets the problem of choosing the best RL algorithm and the best representation of states and actions, while AutoPhase [31] targets the problem of phase ordering, i.e., selecting the best order for compiler passes.

While previous work addresses traditional code optimizations, it does not consider the unique constraints of Fully Homomorphic Encryption. Our work is the first to formulate FHE optimization as an RL problem, where the agent must learn a policy that navigates the trade-offs between vectorization, cryptographic noise accumulation, and the cost of operations.

Tokenization and Variable Renaming. Tokenization has been widely used in NLP. Classical NLP moved from word-level tokenization to learned subword units, with methods such as *Byte-Pair Encoding* (BPE) [54]; more recently, tokenization-free methods process raw characters or bytes (e.g., *CANINE* [16]), which eliminates vocabulary design but lengthens sequences and shifts compute into the model. In coding tasks, BPE has been widely used as a tokenization method for models that take code as input in its textual format (e.g. *CodeBERT* [28]). None of these methods applies variable renaming at the tokenization level, though.

Variable renaming to obtain canonical representations of code was explored for classical clone detection, as in *CCFinder* [35]. Such variable renaming was not used as a tokenization method for deep learning models, though.

Our approach differs from the previous approaches. We normalize the IR by replacing variable names and numeric constants with generic tokens, since their exact values do not affect our rewrite rules, which means that our tokenization method is more specific to our use case. This makes it faster to learn than BPE but still preserves the structural signals the model needs.

Random Code Generation. Random code generation is a common strategy for constructing training corpora for learning-based compiler optimization. Systems such as the Tiramisu autoscheduler [6], Looper [45], and the Halide autoscheduler [1] use stochastic code generators in which the probabilities of syntactic and semantic patterns are manually specified to sample diverse code patterns. To eliminate this manual specification burden, Cummins et al. [18] learn a generative model from large-scale real-world code and then sample synthetic programs whose distribution better matches human-written code. More recently, PIE [55] and EffiCoder [30] use large language models (LLMs) to synthesize code that is subsequently used to train or fine-tune downstream models for performance-oriented code optimization. Our approach follows this latter line: we leverage an off-the-shelf, pretrained LLM, already exposed to the distribution of real-world code, to synthesize training programs, thereby avoiding task-specific generator design and hand-tuning of pattern priors.

H Frequently Asked Questions

In this section, we provide a list of frequently asked questions and answers to these questions. These provide more clarification about the paper.

- **Examples of end-user deployments for homomorphic encryption** End-user deployments for homomorphic encryption have already appeared: for example, Microsoft Edge’s Password Monitor uses homomorphic encryption to check user credentials against breach corpora without revealing inputs [39, 61]. Beyond credentials checking, homomorphic

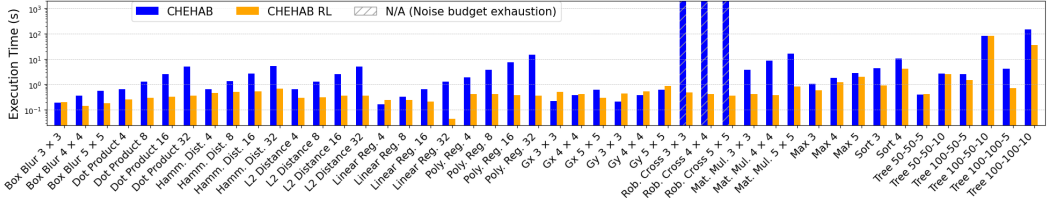


Fig. 8. Semi-log plots comparing the execution times of the default CHEHAB and CHEHAB RL.

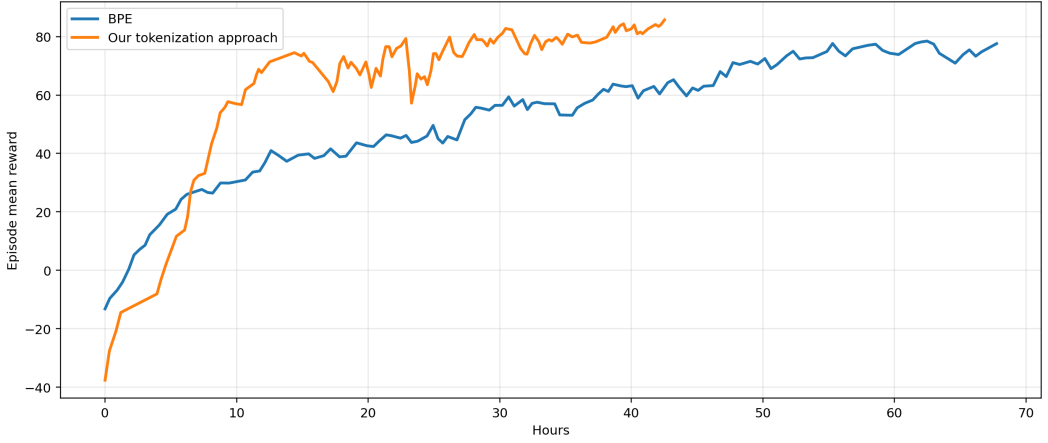


Fig. 9. Graph comparing Episode Mean Reward Over Training Time of the CHEHAB RL with ICI tokenization and CHEHAB RL with BPE (both trained for 2 million steps).

encryption is being used in sensitive scientific/health settings (e.g., homomorphic genotype imputation and encrypted kinship detection in genomics), both reporting practical runtimes on real datasets [22, 37]. The UK Information Commissioner’s Office documents a cross-institution deployment in which banks and law-enforcement agencies use homomorphic encryption to run encrypted queries to detect financial crime [33].

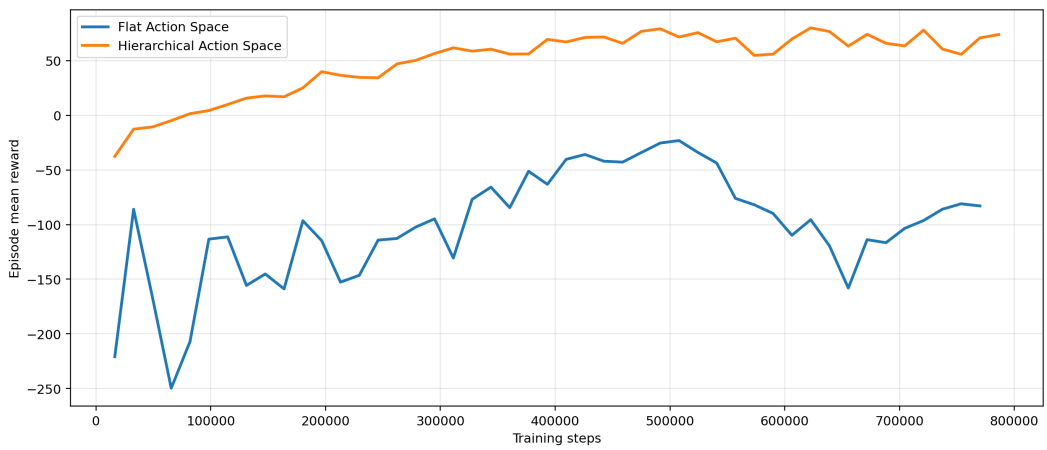


Fig. 10. Graph comparing Episode Mean Reward Over Timesteps of Flat vs. Hierarchical Action Spaces.